



Cloud Computing Lab 14

Submitted To: Sir Waqas Saleem & Sie Shoaib
Submitted By: Hamail Fatima
Roll No: 2023-BSE-023
Section: 5(A)

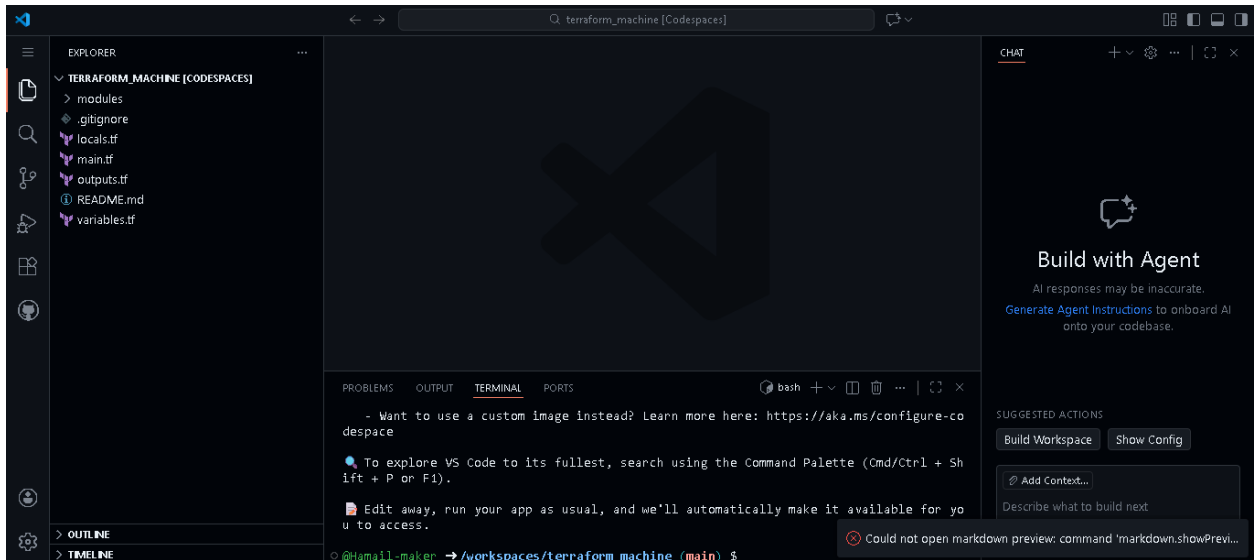
Course Name: Cloud Computing

Lab 14 repo link: https://github.com/Hamail-maker/terraform_machine

Lab # 14

Task 0 - Lab Setup (Codespace & GH CLI)

1. Fork the repo [terraform_machine](#)
2. **Create/open Codespace** on your GitHub account (from terraform_machine repo).
 - **Save screenshot as:** task0_codespace_open.png – browser showing the Codespace opened.



3. Inside the Codespace terminal, verify:
aws --version
terraform --version
ansible --version || echo "ansible not yet installed"
- **Save screenshot as:** task0_env_check.png – Codespace terminal showing aws --version, terraform --version, and ansible --version or the “not installed yet” message.

```

@Hamaill-maker → /workspaces/terraform_machine (main) $ sudo apt install ansible -y
Setting up python3-babel (2.10.3-3build1) ...
update-alternatives: using /usr/bin/pybabel-python3 to provide /usr/bin/pybabel (pybabel) in auto mode
Setting up python3-requests (2.31.0+dfsg-1ubuntu1.1) ...
Setting up python3-netaddr (0.8.0-2ubuntu1) ...
Setting up python3-requests-ntlm (1.1.0-3) ...
Setting up ansible-core (2.16.3-0ubuntu2) ...
Setting up python3-libcloud (3.4.1-5) ...
Setting up python3-winrm (0.4.3-2) ...
Setting up ansible (9.2.0+dfsg-0ubuntu5) ...
Processing triggers for man-db (2.12.0-4build2) ...
● @Hamaill-maker → /workspaces/terraform_machine (main) $ aws --version
terraform --version
ansible --version
aws-cli/2.33.5 Python/3.13.11 Linux/6.8.0-1030-azure exe/x86_64.ubuntu.24
Terraform v1.14.3
on linux_amd64
ansible [core 2.16.3]
  config file = None
  configured module search path = ['/home/codespace/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3/dist-packages/ansible
  ansible collection location = /home/codespace/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/bin/ansible
  python version = 3.12.3 (main, Nov  6 2025, 13:44:16) [GCC 13.3.0] (/usr/bin/python3)
  jinja version = 3.1.6
  libyaml = True
○ @Hamaill-maker → /workspaces/terraform_machine (main) $

```

3. Ensure AWS CLI is configured with credentials that have permissions to create EC2, VPC, subnets, and security groups in region **me-central-1**:

aws sts get-caller-identity

- Save screenshot as: task0_aws_config.png – output of aws sts get-caller-identity.

```

● @Hamaill-maker → /workspaces/terraform_machine (main) $ aws sts get-caller-identity
{
  "UserId": "AIDA42E3QACSNIPKW2KQG",
  "Account": "880794402980",
  "Arn": "arn:aws:iam::880794402980:user/lab-user"
}
○ @Hamaill-maker → /workspaces/terraform_machine (main) $

```

Task 1 - Generate ssh key and Initial Terraform apply

You will start from an existing repository and prepare Terraform variables and SSH keys.

1. Check SSH directory & generate SSH key pair if not already present:

ls ~/.ssh

ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519 -N ""

ls -la ~/.ssh

- Save screenshot as: task1_ssh_keygen_before.png – ls ~/.ssh before key generation.

```

● @Hamaill-maker → /workspaces/terraform_machine (main) $ ls ~/.ssh
ls: cannot access '/home/codespace/.ssh': No such file or directory
○ @Hamaill-maker → /workspaces/terraform_machine (main) $

```

- Save screenshot as: task1_ssh_keygen.png – output of ssh-keygen command.

```
/workspaces/terraform_machine/aws • Contains emphasized items DRTS bash + v [ ] [ ] ... | [ ] [ ] x

@Hamail-maker → /workspaces/terraform_machine (main) $ ls ~/.ssh
ls: cannot access '/home/codespace/.ssh': No such file or directory

@Hamail-maker → /workspaces/terraform_machine (main) $ ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519 -N ""
Generating public/private ed25519 key pair.
Created directory '/home/codespace/.ssh'.
Your identification has been saved in /home/codespace/.ssh/id_ed25519
Your public key has been saved in /home/codespace/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:39AhRx3Jt35v/e4SNrwFv+UM9GgycfsIRYdx+PSAqRY codespace@codespaces-1ab6f7
The key's randomart image is:
+--[ED25519 256]--+
|      .o*      |
|      E.o O.+  |
|      .oo. *o   |
|      o+..=.o   |
|      S.. *. *  |
|      . o+ X.*  |
|      . * @*    |
|      +.B      |
|      ==       |
+-----[SHA256]-----+
@Hamail-maker → /workspaces/terraform_machine (main) $
```

- Save screenshot as: task1_ssh_keygen_after.png — ls -la ~/.ssh showing id_ed25519 and id_ed25519.pub.

```
@Hamail-maker → /workspaces/terraform_machine (main) $ ls -la ~/.ssh
total 20
drwx----- 2 codespace codespace 4096 Jan 23 12:05 .
drwxr-x--- 1 codespace codespace 4096 Jan 23 12:05 ..
-rw----- 1 codespace codespace 419 Jan 23 12:05 id_ed25519
-rw-r--r-- 1 codespace codespace 109 Jan 23 12:05 id_ed25519.pub
@Hamail-maker → /workspaces/terraform_machine (main) $
```

2. Create terraform.tfvars in the repo root:

cd /workspaces/terraform_machine

touch terraform.tfvars

ls -la terraform.tfvars

- Save screenshot as: task1_terraform_tfvars_created.png — touch terraform.tfvars and ls -la terraform.tfvars.

```
-rw-r--r-- 1 codespace codespace 109 Jan 23 12:05 id_ed25519.pub
@Hamail-maker → /workspaces/terraform_machine (main) $ cd /workspaces/terraform_machine
@Hamail-maker → /workspaces/terraform_machine (main) $ touch terraform.tfvars
ls -la terraform.tfvars
-rw-rw-rw- 1 codespace codespace 0 Jan 23 12:08 terraform.tfvars
@Hamail-maker → /workspaces/terraform_machine (main) $
```

Add the following content:

vpc_cidr_block = "10.0.0.0/16"

subnet_cidr_block = "10.0.10.0/24"

availability_zone = "me-central-1a"

env_prefix = "dev"

instance_type = "t3.micro"

public_key = "~/.ssh/id_ed25519.pub"

- `private_key = "~/ssh/id_ed25519"`
- Save screenshot as: `task1_terraform_tfvars.png` – content of `terraform.tfvars`.

```

@Hamail-maker → /workspaces/terraform_machine (main) $ nano terraform.tfvars
@Hamail-maker → /workspaces/terraform_machine (main) $ cat terraform.tfvars
vpc_cidr_block = "10.0.0.0/16"
subnet_cidr_block = "10.0.10.0/24"
availability_zone = "me-central-1a"
env_prefix = "dev"
instance_type = "t3.micro"
public_key = "~/ssh/id_ed25519.pub"
private_key = "~/ssh/id_ed25519"
@Hamail-maker → /workspaces/terraform_machine (main) $

```

3. Initialize Terraform:

`terraform init`

- Save screenshot as: `task1_terraform_init.png` – `terraform init` output.

```

@Hamail-maker → /workspaces/terraform_machine (main) $ terraform init
Initializing the backend...
Initializing modules...
- myapp-subnet in modules/subnet
- myapp-webserver in modules/webserver
Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Finding latest version of hashicorp/http...
- Installing hashicorp/aws v6.28.0...
- Installed hashicorp/aws v6.28.0 (signed by HashiCorp)
- Installing hashicorp/http v3.5.0...
- Installed hashicorp/http v3.5.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
@Hamail-maker → /workspaces/terraform_machine (main) $

```

4. Apply Terraform to create 2 EC2 instances (as defined in the existing Terraform code):

`terraform apply -auto-approve`

- Save screenshot as: `task1_terraform_apply_2_instances.png` – `terraform apply` output showing 2 instances created.

```

@Hamail-maker → /workspaces/terraform_machine (main) $ terraform apply -auto-approve
terraform apply -auto-approve
data.http.my_ip: Reading...
data.http.my_ip: Read complete after 0s [id=https://icanhazip.com]

Terraform used the selected providers to generate the following execution plan. Resource actions are
indicated with the following symbols:
  + create

Terraform will perform the following actions:

# aws_vpc.myapp_vpc will be created
+ resource "aws_vpc" "myapp_vpc" {
  + arn                  = (known after apply)
  + cidr_block           = "10.0.0.0/16"
  + default_network_acl_id = (known after apply)
  + default_route_table_id = (known after apply)
  + default_security_group_id = (known after apply)
  + dhcp_options_id      = (known after apply)
  + enable_dns_hostnames  = true
  + enable_dns_support    = true
  + enable_network_address_usage_metrics = (known after apply)
  + id                   = (known after apply)
  + instance_tenancy      = "default"

```

5. Check outputs:

terraform output

- Save screenshot as: task1_terraform_output_ips.png — terraform output showing instance IPs.

```

@Hamail-maker → /workspaces/terraform_machine (main) $ terraform output
terraform output
webserver_public_ips = [
  "3.28.40.5",
  "158.252.94.173",
]
webserver_public_ips = [
  "3.28.40.5",
  "158.252.94.173",
]
@Hamail-maker → /workspaces/terraform_machine (main) $

```

Task 2 - Static Ansible inventory with two EC2 instances

You will install Ansible (via pipx), create a static inventory, and verify connectivity.

1. Install Ansible (core) using pipx:

pipx install ansible-core

ansible --version

- Save screenshot as: task2_ansible_install.png — pipx install ansible-core and ansible --version output.

```

@Hamail-maker → /workspaces/terraform_machine (main) $ pipx install ansible-core
These apps are now globally available
- ansible
- ansible-config
- ansible-console
- ansible-doc
- ansible-galaxy
- ansible-inventory
- ansible-playbook
- ansible-pull
- ansible-test
- ansible-vault
done! 🌟 🌟 🌟
• @Hamail-maker → /workspaces/terraform_machine (main) $ ansible --version
ansible [core 2.20.1]
  config file = None
  configured module search path = ['/home/codespace/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/local/py-utils/venvs/ansible-core/lib/python3.12/site-packages/ansible
  ansible collection location = /home/codespace/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/local/py-utils/bin/ansible
  python version = 3.12.1 (main, Nov 27 2025, 10:47:52) [GCC 13.3.0] (/usr/local/py-utils/venvs/ansible-core/bin/python)
  jinja version = 3.1.6
  pyyaml version = 6.0.3 (with libyaml v0.2.5)
• @Hamail-maker → /workspaces/terraform_machine (main) $

```

2. Obtain the two public IPs of your EC2 instances:

terraform output

- **Save screenshot as:** task2_terraform_output_ips.png — terraform output listing at least 2 EC2 public IPs.

```

• @Hamail-maker → /workspaces/terraform_machine (main) $ terraform output
webserver_public_ips = [
  "3.28.40.5",
  "158.252.94.173",
]
• @Hamail-maker → /workspaces/terraform_machine (main) $

```

3. Create Ansible inventory file hosts:

touch hosts

ls -la hosts

- **Save screenshot as:** task2_hosts_created.png — confirmation that hosts file exists.

```

• @Hamail-maker → /workspaces/terraform_machine (main) $ touch hosts
ls -la hosts
-rw-rw-rw- 1 codespace codespace 0 Jan 23 12:20 hosts
• @Hamail-maker → /workspaces/terraform_machine (main) $

```

Add the following (replace <public-ip-ec2> with your 2 real IPs):

```

<public-ip-ec2> ansible_user=ec2-user
  ansible_ssh_private_key_file=~/.ssh/id_ed25519

```

```

<public-ip-ec2> ansible_user=ec2-user
  ansible_ssh_private_key_file=~/.ssh/id_ed25519

```

- **Save screenshot as:** task2_hosts_initial.png — content of hosts with two IP lines.

```

• @Hamail-maker → /workspaces/terraform_machine (main) $ cat hosts
3.28.40.5 ansible_user=ec2-user ansible_ssh_private_key_file=~/.ssh/id_ed25519
158.252.94.173 ansible_user=ec2-user ansible_ssh_private_key_file=~/.ssh/id_ed25519
• @Hamail-maker → /workspaces/terraform_machine (main) $

```

4. Test connectivity:

ansible all -i hosts -m ping

- **Save screenshot as:** task2_ansible_ping_initial.png – initial ansible all -i hosts -m ping output (success or failure).

```
@Hamail-maker → /workspaces/terraform_machine (main) $ ansible all -i hosts -m ping
[ERROR]: Task failed: Failed to connect to the host via ssh: Host key verification failed.
Origin: <ad hoc 'ping' task>

{'action': 'ping', 'args': {}, 'timeout': 0, 'async_val': 0, 'poll': 15}

3.28.40.5 | UNREACHABLE! => {
  "changed": false,
  "msg": "Task failed: Failed to connect to the host via ssh: Host key verification failed.",
  "unreachable": true
}
158.252.94.173 | UNREACHABLE! => {
  "changed": false,
  "msg": "Task failed: Failed to connect to the host via ssh: Host key verification failed.",
  "unreachable": true
}
```

5. If it fails due to host key checking, add this to each line in hosts:

ansible_ssh_common_args='-o StrictHostKeyChecking=no'

- **Save screenshot as:** task2_hosts_with_common_args.png – hosts content with ansible_ssh_common_args added.

```
@Hamail-maker → /workspaces/terraform_machine (main) $ cat hosts
3.28.40.5 ansible_user=ec2-user ansible_ssh_private_key_file=~/.ssh/id_ed25519 ansible_ssh_common_arg
s='-o StrictHostKeyChecking=no'
158.252.94.173 ansible_user=ec2-user ansible_ssh_private_key_file=~/.ssh/id_ed25519 ansible_ssh_commo
n_args='-o StrictHostKeyChecking=no'

@Hamail-maker → /workspaces/terraform_machine (main) $
```

6. Retry:

ansible all -i hosts -m ping

- **Save screenshot as:** task2_ansible_ping_success.png – successful ping output.

```
@Hamail-maker → /workspaces/terraform_machine (main) $ ansible all -i hosts -m ping
[WARNING]: Host '3.28.40.5' is using the discovered Python interpreter at '/usr/bin/python3.9', but f
uture installation of another Python interpreter could cause a different interpreter to be disco
vered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for
more information.
3.28.40.5 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.9"
  },
  "changed": false,
  "ping": "pong"
}
[WARNING]: Host '158.252.94.173' is using the discovered Python interpreter at '/usr/bin/python3.9',
but future installation of another Python interpreter could cause a different interpreter to be disco
vered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html
for more information.
158.252.94.173 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.9"
  },
  "changed": false,
  "ping": "pong"
}
```

Task 3 - Scale to three instances & group-based inventory

You will expand to 3 web servers via Terraform's count and restructure your inventory into groups.

1. Update your Terraform module for web servers in main.tf to use count = 3.

```

module "myapp-webserver" {
  source      = "../modules/webserver"
  env_prefix  = var.env_prefix
  instance_type = var.instance_type
  availability_zone = var.availability_zone
  public_key   = var.public_key
  my_ip        = local.my_ip
  vpc_id       = aws_vpc.myapp_vpc.id
  subnet_id    = module.myapp-subnet.subnet.id

  # Loop count
  count       = 3
  # Use count.index to differentiate instances
  instance_suffix = count.index
}

```

- Save screenshot as: task3_main_tf_count_3.png — main.tf snippet showing count = 3.

```

# Loop count
count       = 3

```

2. Apply Terraform to get 3 instances:

terraform apply -auto-approve

- Save screenshot as: task3_terraform_apply_3_instances.png — terraform apply output creating 3 instances.

```

@Hamail-maker → /workspaces/terraform_machine (main) $ terraform apply -auto-approve
~ webserver_public_ips = [
  # (1 unchanged element hidden)
  "158.252.94.173",
  + (known after apply),
]
module.myapp-webserver[2].aws_key_pair.ssh-key: Creating...
module.myapp-webserver[2].aws_security_group.web_sg: Creating...
module.myapp-webserver[2].aws_key_pair.ssh-key: Creation complete after 1s [id=dev-serverkey-2]
module.myapp-webserver[2].aws_security_group.web_sg: Creation complete after 3s [id=sg-0a1353704200ec812]
module.myapp-webserver[2].aws_instance.myapp-server: Creating...
module.myapp-webserver[2].aws_instance.myapp-server: Still creating... [00m10s elapsed]
module.myapp-webserver[2].aws_instance.myapp-server: Creation complete after 13s [id=i-0dbd049ae3785ac9b]

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.

Outputs:

webserver_public_ips = [
  "3.28.40.5",
  "158.252.94.173",
  "158.252.32.2",
]
@Hamail-maker → /workspaces/terraform_machine (main) $

```

3. Check outputs:

terraform output

- Save screenshot as: task3_terraform_output_3_ips.png — terraform output listing 3 public IPs.

```

@Hamail-maker → /workspaces/terraform_machine (main) $ terraform output
webserver_public_ips = [
  "3.28.40.5",
  "158.252.94.173",
  "158.252.32.2",
]
@Hamail-maker → /workspaces/terraform_machine (main) $

```

4. Rewrite your hosts file using group definitions:

[ec2]

<public-ip-ec2>

<public-ip-ec2>

[ec2:vars]

ansible_user=ec2-user

ansible_ssh_private_key_file=~/.ssh/id_ed25519

ansible_ssh_common_args='-o StrictHostKeyChecking=no'

[droplet]

<public-ip-third-ec2>

[droplet:vars]

ansible_user=ec2-user

ansible_ssh_private_key_file=~/.ssh/id_ed25519

ansible_ssh_common_args='-o StrictHostKeyChecking=no'

- Save screenshot as: task3_hosts_grouped.png — hosts showing [ec2] and [droplet] with three IPs.

```

@Hamail-maker → /workspaces/terraform_machine (main) $ cat hosts
[ec2]
3.28.40.5
158.252.94.173

[ec2:vars]
ansible_user=ec2-user
ansible_ssh_private_key_file=~/.ssh/id_ed25519
ansible_ssh_common_args='-o StrictHostKeyChecking=no'

[droplet]
158.252.32.2

[droplet:vars]
ansible_user=ec2-user
ansible_ssh_private_key_file=~/.ssh/id_ed25519
ansible_ssh_common_args='-o StrictHostKeyChecking=no'
@Hamail-maker → /workspaces/terraform_machine (main) $

```

5. Test group connectivity:

ansible ec2 -i hosts -m ping

- Save screenshot as: task3_ansible_ec2_ping.png — output of ansible ec2 -i hosts -m ping.

```

●@Hamail-maker →/workspaces/terraform_machine (main) $ ansible ec2 -i hosts -m ping
[WARNING]: Host '3.28.40.5' is using the discovered Python interpreter at '/usr/bin/python3.9', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
3.28.40.5 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3.9"
    },
    "changed": false,
    "ping": "pong"
}
[WARNING]: Host '158.252.94.173' is using the discovered Python interpreter at '/usr/bin/python3.9', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
158.252.94.173 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3.9"
    },
    "changed": false,
    "ping": "pong"
}

```

6. Test single host by IP:

ansible <one-public-ip-from-ec2-group> -i hosts -m ping

- Save screenshot as: task3_ansible_single_ip_ping.png – ping output for one EC2 IP.

```

●@Hamail-maker →/workspaces/terraform_machine (main) $ ansible 3.28.40.5 -i hosts -m ping
[WARNING]: Host '3.28.40.5' is using the discovered Python interpreter at '/usr/bin/python3.9', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
3.28.40.5 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3.9"
    },
    "changed": false,
    "ping": "pong"
}
●@Hamail-maker →/workspaces/terraform_machine (main) $

```

7. Test droplet group:

ansible droplet -i hosts -m ping

- Save screenshot as: task3_ansible_droplet_ping.png – ping output for droplet group.

```

●@Hamail-maker →/workspaces/terraform_machine (main) $ ansible droplet -i hosts -m ping
[WARNING]: Host '158.252.32.2' is using the discovered Python interpreter at '/usr/bin/python3.9', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
158.252.32.2 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3.9"
    },
    "changed": false,
    "ping": "pong"
}
●@Hamail-maker →/workspaces/terraform_machine (main) $

```

8. Test all hosts:

ansible all -i hosts -m ping

- Save screenshot as: task3_ansible_all_ping.png – ping output for all instances.

```

●@Hamail-maker →/workspaces/terraform_machine (main) $ ansible all -i hosts -m ping
[WARNING]: Host '158.252.32.2' is using the discovered Python interpreter at '/usr/bin/python3.9', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
158.252.32.2 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3.9"
    },
    "changed": false,
    "ping": "pong"
}

```

Task 4 - Global ansible.cfg & first nginx playbook

You will configure a global Ansible configuration file, then create a basic playbook for nginx.

1. Create global Ansible configuration:

vim ~/.ansible.cfg

Add:

[default]

host_key_checking = False

interpreter_python = /usr/bin/python3

- Save screenshot as: task4_global_ansible_cfg.png – content of ~/.ansible.cfg.

```
@Hamail-maker → /workspaces/terraform_machine (main) $ vim ~/.ansible.cfg
@Hamail-maker → /workspaces/terraform_machine (main) $ cat ~/.ansible.cfg
[default]
host_key_checking = False
interpreter_python = /usr/bin/python3
@Hamail-maker → /workspaces/terraform_machine (main) $
```

2. Remove ansible_ssh_common_args from hosts (delete from all groups).

- Save screenshot as: task4_hosts_without_common_args.png – hosts after removal.

```
@Hamail-maker → /workspaces/terraform_machine (main) $ cat hosts
[ec2]
3.28.40.5
158.252.94.173

[ec2:vars]
ansible_user=ec2-user
ansible_ssh_private_key_file=~/.ssh/id_ed25519

[droplet]
158.252.32.2

[droplet:vars]
ansible_user=ec2-user
ansible_ssh_private_key_file=~/.ssh/id_ed25519
@Hamail-maker → /workspaces/terraform_machine (main) $
```

3. Confirm connectivity:

ansible all -i hosts -m ping

- Save screenshot as: task4_ansible_ping_after_cfg.png – ping output showing success, no strict host checking warning.

```
@Hamail-maker → /workspaces/terraform_machine (main) $ ansible all -i hosts -m ping
  },
  "changed": false,
  "ping": "pong"
}
[WARNING]: Host '3.28.40.5' is using the discovered Python interpreter at '/usr/bin/python3.9', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
3.28.40.5 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.9"
  },
  "changed": false,
  "ping": "pong"
}
[WARNING]: Host '158.252.94.173' is using the discovered Python interpreter at '/usr/bin/python3.9', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
158.252.94.173 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.9"
  },
  "changed": false,
  "ping": "pong"
}
@Hamail-maker → /workspaces/terraform_machine (main) $
```

4. Create my-playbook.yaml:

touch my-playbook.yaml

ls -la my-playbook.yaml

- Save screenshot as: task4_my_playbook_created.png – existence of my-playbook.yaml.

```
@Hamail-maker → /workspaces/terraform_machine (main) $ touch my-playbook.yaml
ls -la my-playbook.yaml
-rw-rw-rw- 1 codespace codespace 0 Jan 23 12:49 my-playbook.yaml
@Hamail-maker → /workspaces/terraform_machine (main) $
```

Add:

- name: Configure nginx web server

hosts: ec2

become: true

tasks:

- name: install nginx and update cache

yum:

name: nginx

state: present

update_cache: yes

- name: start nginx server

service:

name: nginx

state: started

- Save screenshot as: task4_my_playbook_ec2.png – content of my-playbook.yaml with hosts: ec2.

```
@Hamail-maker → /workspaces/terraform_machine (main) $ cat my-playbook.yaml
---
- name: Configure nginx web server
  hosts: ec2
  become: true
  tasks:
    - name: install nginx and update cache
      yum:
        name: nginx
        state: present
        update_cache: yes

    - name: start nginx server
      service:
        name: nginx
        state: started
@Hamail-maker → /workspaces/terraform_machine (main) $
```

5. Run the playbook on [ec2] group:

ansible-playbook -i hosts my-playbook.yaml

- Save screenshot as: task4_ansible_play_ec2.png – playbook output for ec2 group.

```
@Hamail-maker → /workspaces/terraform_machine (main) $ ansible-playbook -i hosts my-playbook.yaml
PLAY [Configure nginx web server] *****

TASK [Gathering Facts] *****
[WARNING]: Host '3.28.40.5' is using the discovered Python interpreter at '/usr/bin/python3.9', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
ok: [3.28.40.5]
[WARNING]: Host '158.252.94.173' is using the discovered Python interpreter at '/usr/bin/python3.9', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
ok: [158.252.94.173]

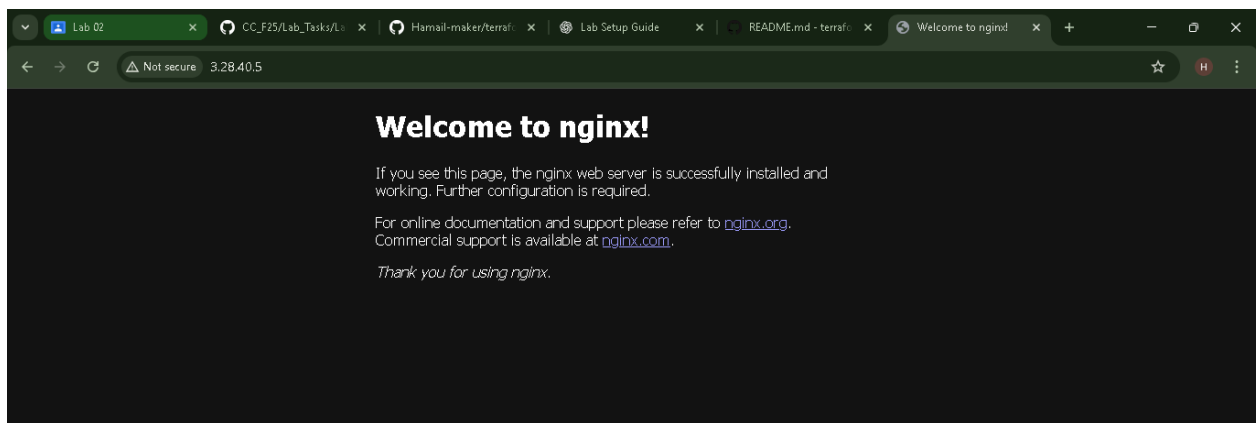
TASK [install nginx and update cache] *****
changed: [3.28.40.5]
changed: [158.252.94.173]

TASK [start nginx server] *****
changed: [3.28.40.5]
changed: [158.252.94.173]

PLAY RECAP *****
158.252.94.173      : ok=3    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
3.28.40.5          : ok=3    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

@Hamail-maker → /workspaces/terraform_machine (main) $
```

6. Verify nginx default page on [ec2] servers by visiting `http://<public-ip-ec2>`.
- Save screenshot as: `task4_nginx_browser_ec2.png` — browser showing nginx default page for an ec2 instance.



7. Change target to droplet:

Edit my-playbook.yaml to:

hosts: droplet

- Save screenshot as: `task4_my_playbook_droplet.png` — playbook with hosts: droplet.

```
● @Hamail-maker → /workspaces/terraform_machine (main) $ nano my-playbook.yaml
● @Hamail-maker → /workspaces/terraform_machine (main) $ cat my-playbook.yaml
---
- name: Configure nginx web server
  hosts: droplet
  become: true
  tasks:
    - name: install nginx and update cache
      yum:
        name: nginx
        state: present
        update_cache: yes

    - name: start nginx server
      service:
        name: nginx
        state: started

● @Hamail-maker → /workspaces/terraform_machine (main) $
```

8. Re-run:

- ansible-playbook -i hosts my-playbook.yaml
- **Save screenshot as:** task4_ansible_play_droplet.png – output of playbook for droplet group.

```
@Hamail-maker → /workspaces/terraform_machine (main) $ ansible-playbook -i hosts my-playbook.yaml

PLAY [Configure nginx web server] *****

TASK [Gathering Facts] *****
[WARNING]: Host '158.252.32.2' is using the discovered Python interpreter at '/usr/bin/python3.9', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
ok: [158.252.32.2]

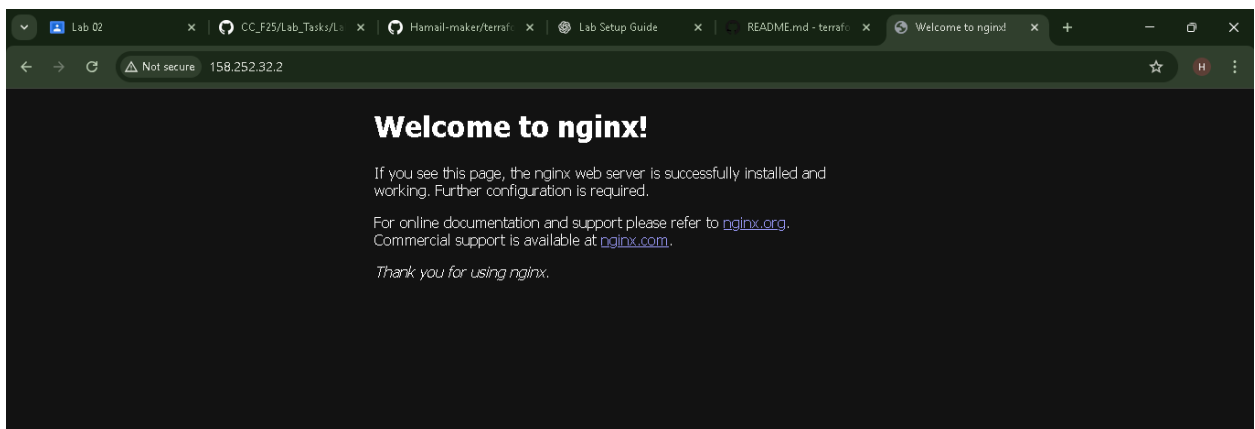
TASK [install nginx and update cache] *****
changed: [158.252.32.2]

TASK [start nginx server] *****
changed: [158.252.32.2]

PLAY RECAP *****
158.252.32.2      : ok=3    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

@Hamail-maker → /workspaces/terraform_machine (main) $
```

9. Verify nginx default page on droplet:
- **Save screenshot as:** task4_nginx_browser_droplet.png – browser showing nginx page on droplet.



Task 5 - Single nginx target group & HTTPS prerequisites

You will prepare project-level Ansible configuration and adjust nginx installation scope.

1. Create project-level ansible.cfg in repo root:
- ```
touch ansible.cfg
ls -la ansible.cfg
```
- **Save screenshot as:** task5\_project\_ansible\_cfg\_created.png – project ansible.cfg presence.

```
@Hamail-maker → /workspaces/terraform_machine (main) $ pwd
/workspaces/terraform_machine
@Hamail-maker → /workspaces/terraform_machine (main) $ touch ansible.cfg
ls -la ansible.cfg
-rw-rw-rw- 1 codespace codespace 0 Jan 23 13:00 ansible.cfg
@Hamail-maker → /workspaces/terraform_machine (main) $
```

Add:

[defaults]

host\_key\_checking=False

- interpreter\_python = /usr/bin/python3
- Save screenshot as: task5\_project\_ansible\_cfg.png – project-level ansible.cfg content.

```
@Hamail-maker → /workspaces/terraform_machine (main) $ nano ansible.cfg
@Hamail-maker → /workspaces/terraform_machine (main) $ cat ansible.cfg
[defaults]
host_key_checking=False
interpreter_python = /usr/bin/python3
@Hamail-maker → /workspaces/terraform_machine (main) $
```

2. Switch Terraform EC2 count back to 1:

In main.tf:

count = 1

- Save screenshot as: task5\_main\_tf\_count\_1.png – snippet showing count = 1.

```
Loop count
count = 1
```

3. Apply:

terraform apply -auto-approve

- Save screenshot as: task5\_terraform\_apply\_one\_instance.png – apply output with one instance.

```
module.myapp.webserver[1].aws_security_group.web_sg: destruction complete after 1s
Apply complete! Resources: 0 added, 0 changed, 6 destroyed.
```

4. Outputs:

terraform output

- Save screenshot as: task5\_terraform\_output\_single\_ip.png – single EC2 public IP.

```
@Hamail-maker → /workspaces/terraform_machine (main) $ terraform output
webserver_public_ips = [
 "3.28.40.5",
]
@Hamail-maker → /workspaces/terraform_machine (main) $
```

5. Adjust hosts:

[nginx]

<single-public-ip>

[nginx:vars]

ansible\_ssh\_private\_key\_file=~/.ssh/id\_ed25519

ansible\_user=ec2-user

- Save screenshot as: task5\_hosts\_nginx\_group.png – hosts file for [nginx].

```
@Hamail-maker → /workspaces/terraform_machine (main) $ cat hosts
[nginx]
3.28.40.5

[nginx:vars]
ansible_ssh_private_key_file=~/.ssh/id_ed25519
ansible_user=ec2-user
@Hamail-maker → /workspaces/terraform_machine (main) $
```

6. Update my-playbook.yaml:

---

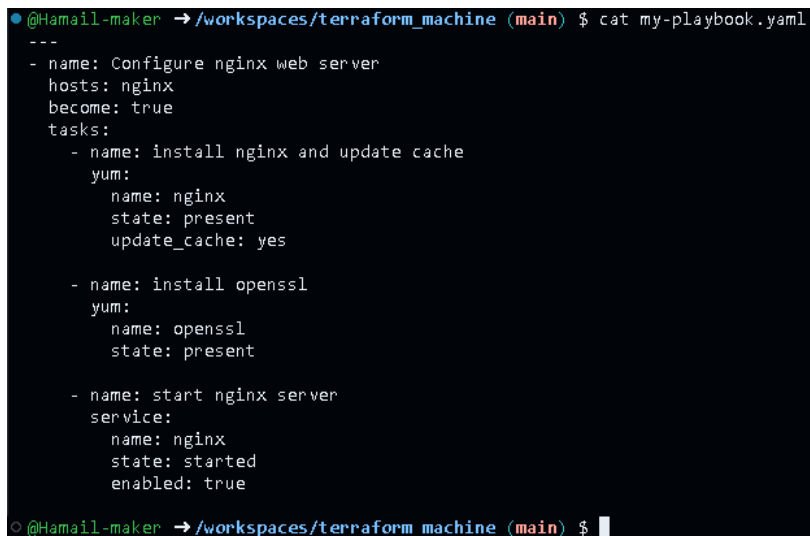
- name: Configure nginx web server

hosts: nginx

become: true

tasks:

- name: install nginx and update cache  
yum:  
  name: nginx  
  state: present  
  update\_cache: yes
  - name: install openssl  
yum:  
  name: openssl  
  state: present
  - name: start nginx server  
service:  
  name: nginx  
  state: started  
  enabled: true
- Save screenshot as: task5\_my\_playbook\_nginx\_group.png – playbook with hosts: nginx.

A terminal window with a dark background. The prompt is '@Hamaill-maker → /workspaces/terraform\_machine (main) \$'. The command 'cat my-playbook.yaml' has been executed, displaying the following YAML content:

```

- name: Configure nginx web server
 hosts: nginx
 become: true
 tasks:
 - name: install nginx and update cache
 yum:
 name: nginx
 state: present
 update_cache: yes

 - name: install openssl
 yum:
 name: openssl
 state: present

 - name: start nginx server
 service:
 name: nginx
 state: started
 enabled: true
```

The terminal cursor is at the end of the prompt line.

7. Run the playbook:  
ansible-playbook -i hosts my-playbook.yaml

- Save screenshot as: task5\_ansible\_play\_nginx\_group.png – playbook output.

```

@Hamall-maker → /workspaces/terraform_machine (main) $ ansible-playbook -i hosts my-playbook.yaml
[WARNING]: Ansible is being run in a world writable directory (/workspaces/terraform_machine), ignoring it as an ansible.cfg source. For more information see https://docs.ansible.com/ansible/devel/reference_appendices/config.html#cfg-in-world-writable-dir

PLAY [Configure nginx web server] *****

TASK [Gathering Facts] *****
[WARNING]: Host '3.28.40.5' is using the discovered Python interpreter at '/usr/bin/python3.9', but future installations of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
ok: [3.28.40.5]

TASK [install nginx and update cache] *****
ok: [3.28.40.5]

TASK [install openssl] *****
ok: [3.28.40.5]

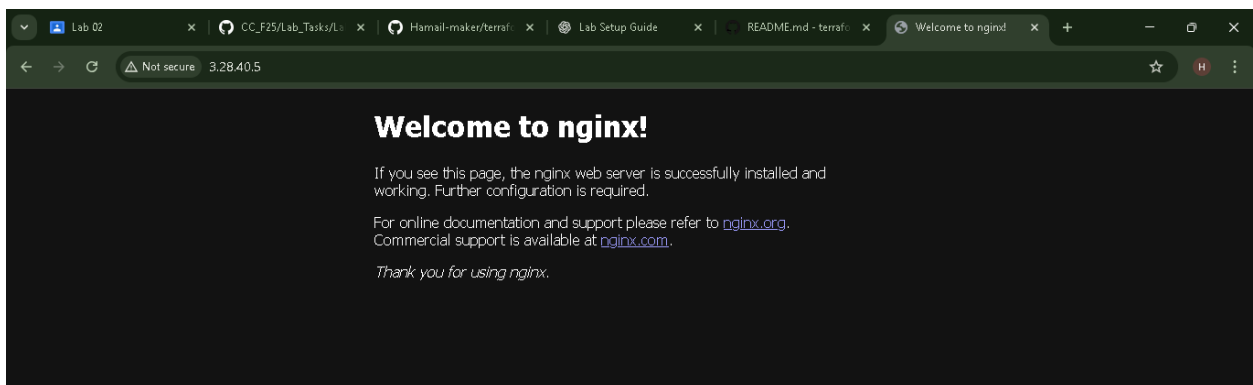
TASK [start nginx server] *****
changed: [3.28.40.5]

PLAY RECAP *****
3.28.40.5 : ok=4 changed=1 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0

@Hamall-maker → /workspaces/terraform_machine (main) $

```

8. Verify nginx default page at `http://<public-ip>`.
- Save screenshot as: `task5_nginx_browser_single.png` – browser view.



## Task 6 - Ansible-managed SSL certificates

Extend your playbook to generate **self-signed SSL certificates** using dynamic public IP.

1. Append this play after the nginx play in my-playbook.yaml:

- name: Configure SSL certificates
  - hosts: nginx
  - become: true
  - tasks:
    - name: Create SSL private directory
      - file:
        - path: /etc/ssl/private
        - state: directory
        - mode: '0700'
    - name: Create SSL certs directory
      - file:
        - path: /etc/ssl/certs
        - state: directory
        - mode: '0755'
    - name: Get IMDSv2 token
      - uri:

```
url: "http://169.254.169.254/latest/api/token"
method: PUT
headers:
 X-aws-ec2-metadata-token-ttl-seconds: "3600"
return_content: yes
register: imdsv2_token
```

```
- name: Get current public IP
uri:
 url: "http://169.254.169.254/latest/meta-data/public-ipv4"
 headers:
 X-aws-ec2-metadata-token: "{{ imdsv2_token.content }}"
 return_content: yes
register: public_ip
```

```
- name: Show current public IP
debug:
 msg: "Public IP: {{ public_ip.content }}"
```

```
- name: Generate self-signed SSL certificate
command: >
 openssl req -x509 -nodes -days 365
 -newkey rsa:2048
 -keyout /etc/ssl/private/selfsigned.key
 -out /etc/ssl/certs/selfsigned.crt
 -subj "/CN={{ public_ip.content }}"
 -addext "subjectAltName=IP:{{ public_ip.content }}"
 -addext "basicConstraints=CA:FALSE"
 -addext "keyUsage=digitalSignature,keyEncipherment"
 -addext "extendedKeyUsage=serverAuth"
args:
 creates: /etc/ssl/certs/selfsigned.crt
```

- **Save screenshot as:** task6\_my\_playbook\_ssl\_section.png – SSL block in my-playbook.yaml.

```
@Hamaill-maker → /workspaces/terraform_machine (main) $ cat my-playbook.yaml
 return_content: yes
 register: imdsv2_token

- name: Get current public IP
 uri:
 url: "http://169.254.169.254/latest/meta-data/public-ipv4"
 headers:
 X-aws-ec2-metadata-token: "{{ imdsv2_token.content }}"
 return_content: yes
 register: public_ip

- name: Show current public IP
 debug:
 msg: "Public IP: {{ public_ip.content }}"

- name: Generate self-signed SSL certificate
 command: >
 openssl req -x509 -nodes -days 365
 -newkey rsa:2048
 -keyout /etc/ssl/private/selfsigned.key
 -out /etc/ssl/certs/selfsigned.crt
 -subj "/CN={{ public_ip.content }}"
 -addext "subjectAltName=IP:{{ public_ip.content }}"
 -addext "basicConstraints=CA:FALSE"
 -addext "keyUsage=digitalSignature,keyEncipherment"
 -addext "extendedKeyUsage=serverAuth"
 args:
 creates: /etc/ssl/certs/selfsigned.crt

@Hamaill-maker → /workspaces/terraform_machine (main) $
```

## 2. Run:

ansible-playbook -i hosts my-playbook.yaml

- ```
@Hamail-maker → /workspaces/terraform_machine (main) $ ansible-playbook -i hosts my-playbook.yaml

PLAY [Configure SSL certificates] *****

TASK [Gathering Facts] *****
ok: [3.28.40.5]

TASK [Create SSL private directory] *****
changed: [3.28.40.5]

TASK [Create SSL certs directory] *****
changed: [3.28.40.5]

TASK [Get IMDSv2 token] *****
ok: [3.28.40.5]

TASK [Get current public IP] *****
ok: [3.28.40.5]

TASK [Show current public IP] *****
ok: [3.28.40.5] => {
  "msg": "Public IP: 3.28.40.5"
}

TASK [Generate self-signed SSL certificate] *****
changed: [3.28.40.5]

PLAY RECAP *****
3.28.40.5          : ok=11   changed=3    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

@Hamail-maker → /workspaces/terraform_machine (main) $
```

```
ssh ec2-user@<public-ip> -i ~/.ssh/id_ed25519
```

```
sudo cat /etc/ssl/private/selfsigned.key
```

Save screenshot as: task6_ssl_cert_file.png — cat /etc/ssl/certs/selfsigned.crt.

```
@Hamail-maker → /workspaces/terraform_machine (main) $ ssh ec2-user@3.28.40.5 -i ~/.ssh/id_ed25519
```

```
~
~ \###|
~ \#/ https://aws.amazon.com/linux/amazon-linux-2023
~ \w*'-'>
~
~ _.'
~ _.'
~ _./'
~ _./'
```

```
Last login: Fri Jan 23 13:16:21 2026 from 20.192.21.51
[ec2-user@ip-10-0-10-184 ~]$ sudo cat /etc/ssl/certs/selfsigned.crt
-----BEGIN CERTIFICATE-----
MIIDNzCCAah+gAwIBAgIUIT6Mj53xR5ehUR7EnN/Ms7SmEKmWdQYJKoZIhvcNAQEL
BQAwFDESMBAGA1UEAwwJMy4yOC40MC41MB4XDTI2MDEyMzEzMTYyMVoXDTE3MDEy
MzEzMTYyMVoMFDESMBAGA1UEAwwJMy4yOC40MC41MIIBIjANBgkqhkiG9w0BAQEFAA
OCAQA8AMIIBCgKCAQEAmpRS851Mw35Nlw+8gNdffPzytCsmoxkJAUSATAiF8PtIF
cr+n/7jihkQwAei+1y1RV6zVDL2ehHvEI6GXh1EWJTniLnLMJ2SNL4ZqYNurGuho
f9yo/JIfweZBPftI8mmBzd2kyeA/ovb5XSzmQSHOATrrNRn3ozl8os5ks1FuZrdl
RPuFW8uY3CDfffCHuoFug7RDttP2ABP7EAaq/dMr588Pcw9ETwa/5m7xo2bk5ou1
ktzbBYLjq5oiXNhKiZ7RaSMm2RbcZLkLw5tralJA5xor7A20fixmDmbbi/reIsXp
e1uCumyA4hcczmZYDwmcdc2rDls1SnZxaYVvkG5xAwIDAQABo4GA MH4wHQYDVRR0
BBVEFFo/YcNCYKDaxCpI/HXKEFMva0gMB8GA1UdIwQYMBAAFFo/YcNCYKDaxCpI
/HXKEFMva0gMA8GA1UdeQQIMAaHBAMcKAUwCQYDVROTBAlwADALBgNVHQ8EBAMC
BaAlwEuYDYVR0LBAlwCcGYIKwYBBQUHAWEwDQYJKoZIhvcNAQELBQADggEBAA/rjkEP
3LAALFmYAxtW5errTHLbbFvwaaFY/+NfqFMQq4ZittOGKg0chiNBln+me7AQST6H
FSQDwsNp2VNw-JcgOTpsD/oBpzIKCAiuq2alr3stx1v+2E vBSQElyzoppjXK/Rwe5
5GR18CxY5y7OGHQI7YNOWkvynOH7Mqr1FO/SofGAR0e3YMHkh2YpQT7qWgRJpp
3KUJz9q8tXuXMO82RRqZsr1v64Yb+cDCMZE7udsKRXVfaw1n0mbQwTIRB321hc
g3mbzW3fIISWvHaLfzwYqvruoNONkAY9sSB8bj1HnIAAXE76oVLl82yx6PrphayK
Yiv+qfZxLk1qFcA=
-----END CERTIFICATE-----
[ec2-user@ip-10-0-10-184 ~]$ █
```

- **Save screenshot as:** `task6_ssl_key_file.png` – `cat /etc/ssl/private/selfsigned.key`.

```
@Hamail-maker → /workspaces/terraform_machine (main) $ ssh ec2-user@3.28.40.5 -i ~/.ssh/id_ed25519
[ec2-user@ip-10-0-10-184 ~]$ sudo cat /etc/ssl/private/selfsigned.key
-----BEGIN PRIVATE KEY-----
MIIEvQIBADANBgkqhkiG9w0BAQEFAASCBAQwggSjAgEAAoIBAQCaulLzmIzDfk3D
7yA118/P1NNxKaJG5MBRIBMCIXy1MgVyyv6f/uOKGQLAB6L7XLVFXrNUOXZ6E4V4j
oZeHURYlOeIusswnZI0vhmpg26sa6E5/3Kj8kh/B5kE9+0jyabzN3aRh4D+i9vld
LOZBIc4BOus1GfehmxzSzmSyUW5mt2VE+4Vby5jcIN98Vwe6h9SDtEN00/YAE/sR
oCr90yvnzW9xb0RPBr/mbvE7ZuTm17W53MHxguOrk6Jc2EgJntFpIwzZFsJkuQvD
m2tqWMBLGHsDbr+LGYOaZuL+t4ixel7W4K6bIDiFxzObNgPCZ1zasOWyVKnFrJ
W+QbnEBbAgMBAAECggEAO254MFMXNnwMOmdf8u6LRsOsD9p1lnwZSs/sHTRpoTZe
e138lLaaoDsWj9hFlu0091yFMuG+M/wMfV+s55EYfW8eD17R2JLDMcVvkzatOW+j
v18t1Z43Do533u vncB2HRPfuJR3Ks7jz vYNIigAHFdfgwoVP30WkeZqHeSr oIHGU
IRvsqy81yfwcdXyNqdmhA0JqfwLM/w5T vX6c7EYKt1hJ3WUvMq/gOALbEWQYC2yO
M+c6hwK0mNXRwn vVRSR5PfoXXi1bzCVCroMBvt4K6uMqG3nf7UhmC4Pcs17q85Q5
KIFowt1tX0cNaER473rp7WwFjUQRf7FmWrZLwa/yQKBgQDY8hZZq70BaY4vb10U
OwYFqdgwmgK8/z8k0dxt drTok/vcSLG03biNMOSYWPwFFVuML60563czUZXSAts
1YEhXd6+R2Uk v77PBBvNBxWu27RLn vx9FN0TJXC8631Uh8aY9GHv3WrCBWQ7xku
xE dxq1gBIqOyyz24bWo vWjBw9wKBgQC2L0+rABRW2mqEaTXNtbtEa3TI0oEZGWF
ApXEjRP2G6n+MFOwN4VZUw1ip7Def6bywtzOpjTMamu2cQ1DOioXOG/ZiMm6bfWP
n2VrS/xh18Up1NeUwbtAMuVnH4yE+kj4V8ONGu/II1NB+OvAwVkyNPvWvJqhkGix
onIUabF2vQKBgD4u/esMBTL176R3E4zb/Ng/O8GWCrtJSEHsx0L1EJvNtXaIG6/N
EdYempd0gErue0Swd+1N4CVNRjrtiSm+hF3V55Q1p3Bh1Ogy18h2GMhxa40KT
RMb1J2BvwmK5XFqBwUJgw/YDEtXrQtud1ScmdkJ451UcXGueaR5xWfM3AoGAeEQY
rtnp1+BpY3juA0ehSBOQf7ByE/yRxpCWoK0Rjfyk7N0ma3987f60Yue0oIsTS/bA
FbrzcmzbNoS35L4qdYsuWes5ffzqUMWdB5VQ1HQ1YPxZWJXWuW4Lf+cPJ555tiQR
Ys1165hmm6E8oY26m5C9BpDLhnvsEB5B60JKDD0CgYEAqA9kuwjr qVnrS8uJozZE
b9kVGUNWEmmhQI3E8rUqcsEVSTLIN5auD2Q9CgZad6Ek94s2w4hw4M0dkgj8Xpo
urKALiMOyUEpTk3+ss12ph9jpe2J6XJwf/PtnPROJbWw9Rcv+YmSHoUwmNwPEIza
5BIO3m7xQjv1w1IqRiWkEzE=
-----END PRIVATE KEY-----
[ec2-user@ip-10-0-10-184 ~]$
```

Task 7 - PHP front-end deployment with templates

Deploy a PHP application and Nginx configuration using Ansible copy and template.

1. Create directories and files:
 mkdir -p files templates
 touch files/index.php
 touch templates/nginx.conf.j2
 ls -R
- Save screenshot as: task7_files_templates_created.png – ls -R showing files/index.php and templates/nginx.conf.j2.

```
./aws/dist/setuptools:
_vendor

./aws/dist/setuptools/_vendor:
jaraco

./aws/dist/setuptools/_vendor/jaraco:
text

./aws/dist/setuptools/_vendor/jaraco/text:
'Lorem ipsum.txt'

./aws/dist/wheel-0.45.1.dist-info:
INSTALLER LICENSE.txt METADATA RECORD REQUESTED WHEEL direct_url.json entry_points.txt

./files:
index.php

./modules:
subnet webserver

./modules/subnet:
main.tf outputs.tf variables.tf

./modules/webserver:
main.tf outputs.tf variables.tf

./templates:
nginx.conf.j2
@Hamail-maker → /workspaces/terraform_machine (main) $
```

2. Fill files/index.php with the following PHP metadata page:

```
<?php
// Get hostname
$hostname = gethostname();

// Deployment date
$deployed_date = date("Y-m-d H:i:s");

// Metadata base URL
$metadata_base = "http://169.254.169.254/latest/";

// Function to get IMDSv2 token
function getImdsV2Token() {
    $ch = curl_init("http://169.254.169.254/latest/api/token");
    curl_setopt_array($ch, [
        CURLOPT_RETURNTRANSFER => true,
        CURLOPT_CUSTOMREQUEST => "PUT",
        CURLOPT_HTTPHEADER => [
            "X-aws-ec2-metadata-token-ttl-seconds: 21600"
        ],
        CURLOPT_TIMEOUT => 2
    ]);

    $token = curl_exec($ch);
    curl_close($ch);

    return $token ?: null;
}

// Function to fetch metadata using token
function getMetadata($path, $token) {
    $url = "http://169.254.169.254/latest/meta-data/" . $path;

    $ch = curl_init($url);
    curl_setopt_array($ch, [
        CURLOPT_RETURNTRANSFER => true,
        CURLOPT_HTTPHEADER => [
            "X-aws-ec2-metadata-token: $token"
        ],
        CURLOPT_TIMEOUT => 2
    ]);

    $value = curl_exec($ch);
    curl_close($ch);

    return $value ?: "N/A";
}

// Fetch token
$token = getImdsV2Token();

// Fetch metadata only if token is available
$instance_id = $token ? getMetadata("instance-id", $token) : "N/A";
$private_ip = $token ? getMetadata("local-ipv4", $token) : "N/A";
$public_ip = $token ? getMetadata("public-ipv4", $token) : "N/A";
```

```

$public_dns = $token ? getMetadata("public-hostname", $token) : "N/A";
?>
<!DOCTYPE html>
<html>
<head>
  <title>Frontend Web Server</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 50px;
      background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
      color: white;
    }
    .container {
      background: rgba(255, 255, 255, 0.1);
      padding: 30px;
      border-radius: 10px;
      box-shadow: 0 8px 32px rgba(31, 38, 135, 0.37);
    }
    h1 {
      color: #fff;
      text-shadow: 2px 2px 4px rgba(0,0,0,0.3);
    }
    .info {
      margin: 15px 0;
      padding: 10px;
      background: rgba(255,255,255,0.2);
      border-radius: 5px;
    }
    .label {
      font-weight: bold;
      color: #ffd700;
    }
    .info a {
      color: white;          /* same as other values */
      text-decoration: none; /* remove underline */
      font-weight: normal;
    }
    .info a:hover {
      text-decoration: underline; /* optional: underline on hover */
    }
  </style>
</head>
<body>
  <div class="container">
    <h1>🚀 Nginx Front End Web Server </h1>

    <div class="info"><span class="label">Hostname:</span> <?=
htmlspecialchars($hostname) ?></div>
    <div class="info"><span class="label">Instance ID:</span> <?=
htmlspecialchars($instance_id) ?></div>
    <div class="info"><span class="label">Private IP:</span> <?=
htmlspecialchars($private_ip) ?></div>
    <div class="info"><span class="label">Public IP:</span> <?=

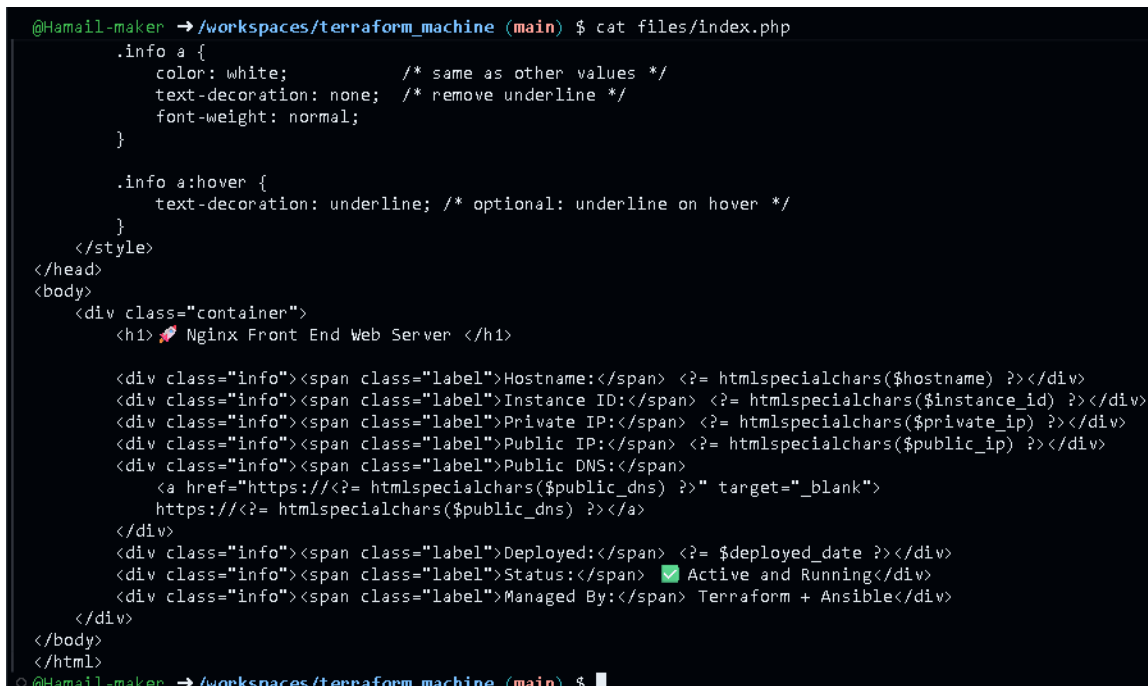
```

```

htmlspecialchars($public_ip) ?></div>
    <div class="info"><span class="label">Public DNS:</span>
        <a href="https://<?= htmlspecialchars($public_dns) ?>" target="_blank">
            https://<?= htmlspecialchars($public_dns) ?></a>
        </div>
    <div class="info"><span class="label">Deployed:</span> <?= $deployed_date ?></div>
    <div class="info"><span class="label">Status:</span> ☒ Active and Running</div>
    <div class="info"><span class="label">Managed By:</span> Terraform + Ansible</div>
</div>
</body>
</html>

```

- Save screenshot as: task7_index_php_content.png – content of files/index.php in editor or cat.



```

@Hamail-maker → /workspaces/terraform_machine (main) $ cat files/index.php
<?php
    .info a {
        color: white;           /* same as other values */
        text-decoration: none; /* remove underline */
        font-weight: normal;
    }

    .info a:hover {
        text-decoration: underline; /* optional: underline on hover */
    }
</style>
</head>
<body>
    <div class="container">
        <h1>🔥 Nginx Front End Web Server </h1>

        <div class="info"><span class="label">Hostname:</span> <?= htmlspecialchars($hostname) ?></div>
        <div class="info"><span class="label">Instance ID:</span> <?= htmlspecialchars($instance_id) ?></div>
        <div class="info"><span class="label">Private IP:</span> <?= htmlspecialchars($private_ip) ?></div>
        <div class="info"><span class="label">Public IP:</span> <?= htmlspecialchars($public_ip) ?></div>
        <div class="info"><span class="label">Public DNS:</span>
            <a href="https://<?= htmlspecialchars($public_dns) ?>" target="_blank">
                https://<?= htmlspecialchars($public_dns) ?></a>
            </div>
        <div class="info"><span class="label">Deployed:</span> <?= $deployed_date ?></div>
        <div class="info"><span class="label">Status:</span> ☒ Active and Running</div>
        <div class="info"><span class="label">Managed By:</span> Terraform + Ansible</div>
    </div>
</body>
</html>
@Hamail-maker → /workspaces/terraform_machine (main) $

```

3. Fill templates/nginx.conf.j2 with the following Nginx configuration template:

```

user nginx;
worker_processes auto;
error_log /var/log/nginx/error.log notice;
pid /run/nginx.pid;

events {
    worker_connections 1024;
}

http {
    log_format main '$remote_addr - $remote_user [$time_local] "$request"
        '$status $body_bytes_sent "$http_referer"
        "'$http_user_agent" "$http_x_forwarded_for"';

    access_log /var/log/nginx/access.log main;

    sendfile        on;
    tcp_nopush      on;
    keepalive_timeout 65;

```

```
types_hash_max_size 4096;
```

```
include      /etc/nginx/mime.types;  
default_type application/octet-stream;
```

```
upstream backend_servers {  
    server 158.252.94.241:80;  
    server 158.252.94.242:80 backup;  
}
```

```
server {  
    listen 443 ssl;  
    server_name {{ server_public_ip }};  
    ssl_certificate /etc/ssl/certs/selfsigned.crt;  
    ssl_certificate_key /etc/ssl/private/selfsigned.key;
```

```
    location / {  
        root /usr/share/nginx/html;  
        index index.php index.html index.htm;  
        # proxy_pass http://158.252.94.241:80;  
        # proxy_pass http://backend_servers;
```

```
        #  This block is necessary for Php Website
```

```
        location ~ \.php$ {  
            include fastcgi_params;  
            fastcgi_pass unix:/run/php-fpm/www.sock;  
            fastcgi_index index.php;  
            fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;  
        }  
    }  
}
```

```
server {  
    listen 80;  
    server_name _;  
    return 301 https://$host$request_uri;  
}
```

- Save screenshot as: task7_nginx_conf_template.png – content of templates/nginx.conf.j2.

```
@Hamail-maker → /workspaces/terraform_machine (main) $ cat templates/nginx.conf.j2

server {
    listen 443 ssl;
    server_name {{ server_public_ip }};
    ssl_certificate /etc/ssl/certs/selfsigned.crt;
    ssl_certificate_key /etc/ssl/private/selfsigned.key;

    location / {
        root /usr/share/nginx/html;
        index index.php index.html index.htm;
        # proxy_pass http://158.252.94.241:80;
        # proxy_pass http://backend_servers;

        # 🟡 This block is necessary for Php Website
        location ~ \.php$ {
            include fastcgi_params;
            fastcgi_pass unix:/run/php-fpm/www.sock;
            fastcgi_index index.php;
            fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
        }
    }
}

server {
    listen 80;
    server_name _;
    return 301 https://$host$request_uri;
}
}

@Hamail-maker → /workspaces/terraform_machine (main) $
```

4. Add this play to my-playbook.yaml:

- name: Deploy Nginx website and configuration files

hosts: nginx

become: true

tasks:

- name: install php-fpm and php-curl

yum:

name:

- php-fpm

- php-curl

state: present

- name: Copy website files

copy:

src: files/index.php

dest: /usr/share/nginx/html/index.php

owner: nginx

group: nginx

mode: '0644'

- name: Copy nginx.conf template

template:

src: templates/nginx.conf.j2

dest: /etc/nginx/nginx.conf

owner: root

group: root

mode: '0644'

- name: Restart nginx

service:

name: nginx

state: restarted

- name: Start and enable php-fpm

```
service:
  name: php-fpm
  state: started
  enabled: true
```

- Save screenshot as: task7_my_playbook_web_deploy.png – updated my-playbook.yaml including this play.

```
@Hamail-maker → /workspaces/terraform_machine (main) $ cat my-playbook.yaml
- php-curl
  state: present

- name: Copy website files
  copy:
    src: files/index.php
    dest: /usr/share/nginx/html/index.php
    owner: nginx
    group: nginx
    mode: '0644'

- name: Copy nginx.conf template
  template:
    src: templates/nginx.conf.j2
    dest: /etc/nginx/nginx.conf
    owner: root
    group: root
    mode: '0644'

- name: Restart nginx
  service:
    name: nginx
    state: restarted

- name: Start and enable php-fpm
  service:
    name: php-fpm
    state: started
    enabled: true

@Hamail-maker → /workspaces/terraform_machine (main) $
```

5. Run the playbook:

ansible-playbook -i hosts my-playbook.yaml

- Save screenshot as: task7_ansible_play_web_deploy.png – playbook output showing PHP/nginx tasks executed.

```
@Hamail-maker → /workspaces/terraform_machine (main) $ ansible-playbook -i hosts my-playbook.yaml
"msg": "Public IP: 3.28.40.5"
}

TASK [Generate self-signed SSL certificate] *****
ok: [3.28.40.5]

PLAY [Deploy Nginx website and configuration files] *****

TASK [Gathering Facts] *****
ok: [3.28.40.5]

TASK [install php-fpm and php-curl] *****
ok: [3.28.40.5]

TASK [Copy website files] *****
ok: [3.28.40.5]

TASK [Copy nginx.conf template] *****
changed: [3.28.40.5]

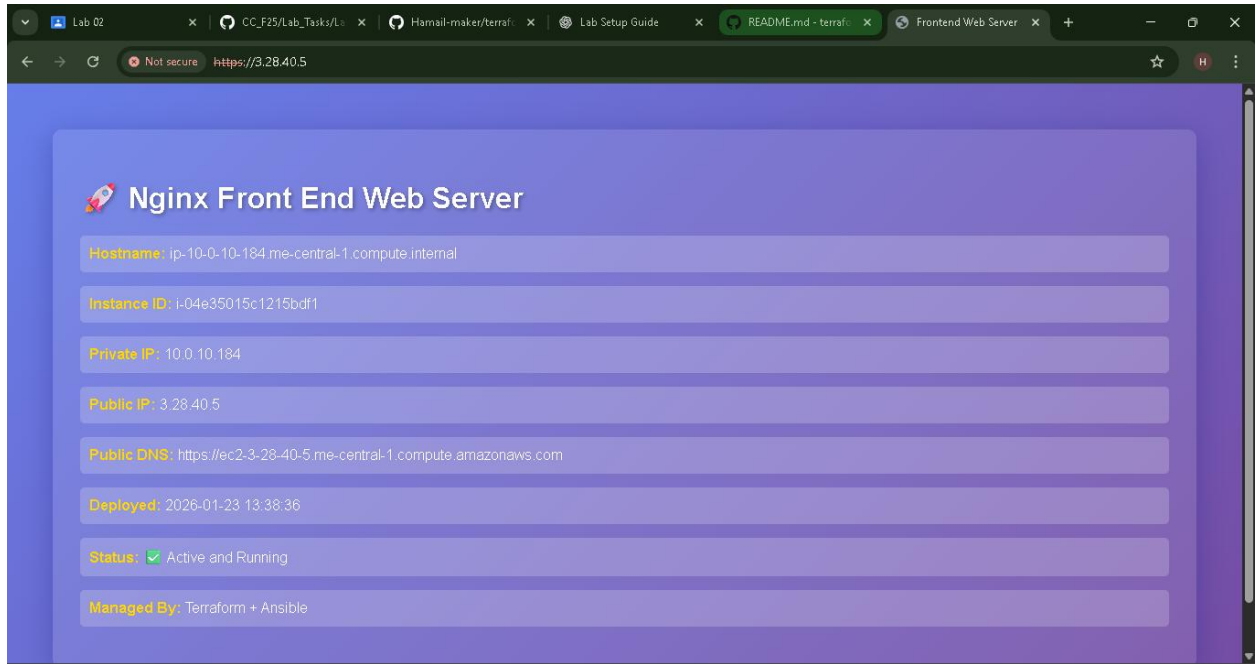
TASK [Restart nginx] *****
changed: [3.28.40.5]

TASK [Start and enable php-fpm] *****
changed: [3.28.40.5]

PLAY RECAP *****
3.28.40.5 : ok=17  changed=3  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0

@Hamail-maker → /workspaces/terraform_machine (main) $
```

6. Visit `https://<public-ip>`:
 - Accept SSL warning.
 - Verify the PHP page content.
- Save screenshot as: `task7_php_https_browser.png` – browser showing PHP web page via HTTPS.



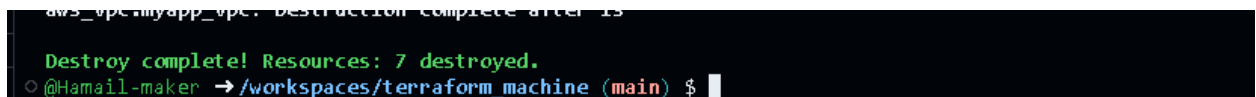
Task 8 - Docker & Docker Compose provisioning via Ansible

Deploy Docker and Docker Compose on a new EC2 instance.

1. Destroy old infrastructure:

`terraform destroy -auto-approve`

- Save screenshot as: `task8_terraform_destroy_old.png` – destroy output for old stack.

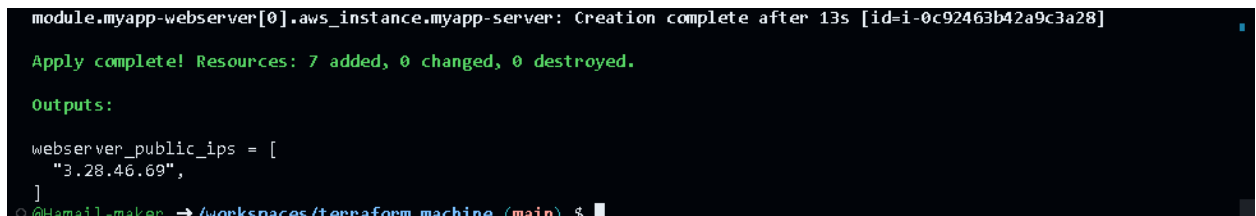


2. Recreate fresh infrastructure (1 instance):

`terraform apply -auto-approve`

`terraform output`

- Save screenshot as: `task8_terraform_apply_docker_instance.png` – new apply.



- Save screenshot as: `task8_terraform_output_new_ip.png` – output showing new instance IP.

```

@Hamail-maker → /workspaces/terraform_machine (main) $ terraform output
webserver_public_ips = [
  "3.28.46.69",
]
@Hamail-maker → /workspaces/terraform_machine (main) $

```

3. Update hosts:

[docker_servers]

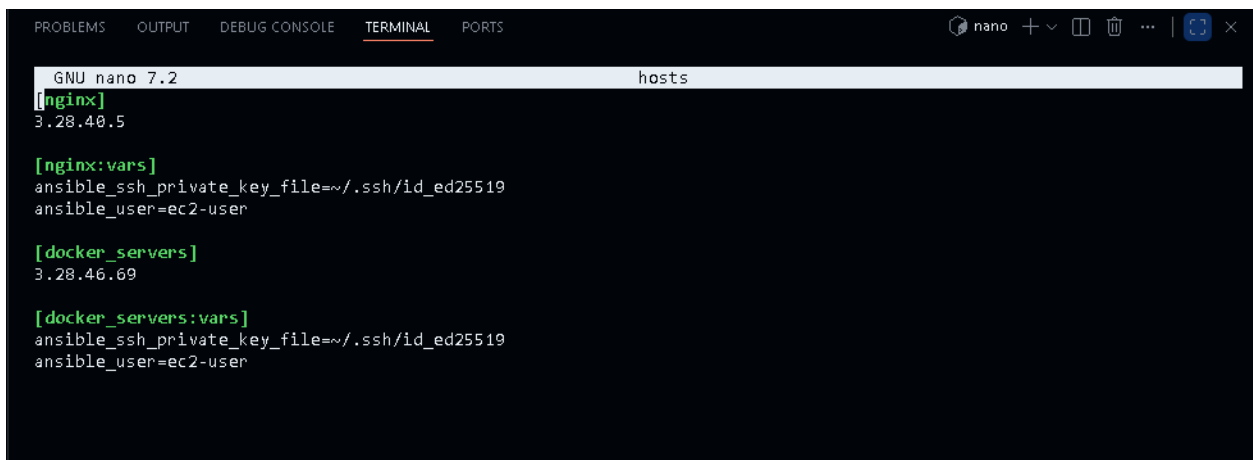
<public-ip-new-ec2>

[docker_servers:vars]

ansible_ssh_private_key_file=~/.ssh/id_ed25519

ansible_user=ec2-user

- **Save screenshot as:** task8_hosts_docker_servers.png – hosts file showing [docker_servers].



```

GNU nano 7.2 hosts
[nginx]
3.28.40.5

[nginx:vars]
ansible_ssh_private_key_file=~/.ssh/id_ed25519
ansible_user=ec2-user

[docker_servers]
3.28.46.69

[docker_servers:vars]
ansible_ssh_private_key_file=~/.ssh/id_ed25519
ansible_user=ec2-user

```

4. Replace my-playbook.yaml content with:

- name: Configure Docker
 - hosts: all
 - become: true
 - tasks:
 - name: install docker and update cache
 - yum:
 - name: docker
 - state: present
 - update_cache: yes
- name: Install Docker Compose
 - hosts: all
 - become: true
 - gather_facts: true
 - tasks:
 - name: create docker cli-plugins directory
 - file:
 - path: /usr/local/lib/docker/cli-plugins
 - state: directory
 - mode: '0755'
 - name: install docker-compose
 - get_url:
 - url: <https://github.com/docker/compose/releases/latest/download/docker->

url: <https://github.com/docker/compose/releases/latest/download/docker->

```

compose-linux-{{ lookup('pipe', 'uname -m') }}
  dest: /usr/local/lib/docker/cli-plugins/docker-compose
  mode: +x

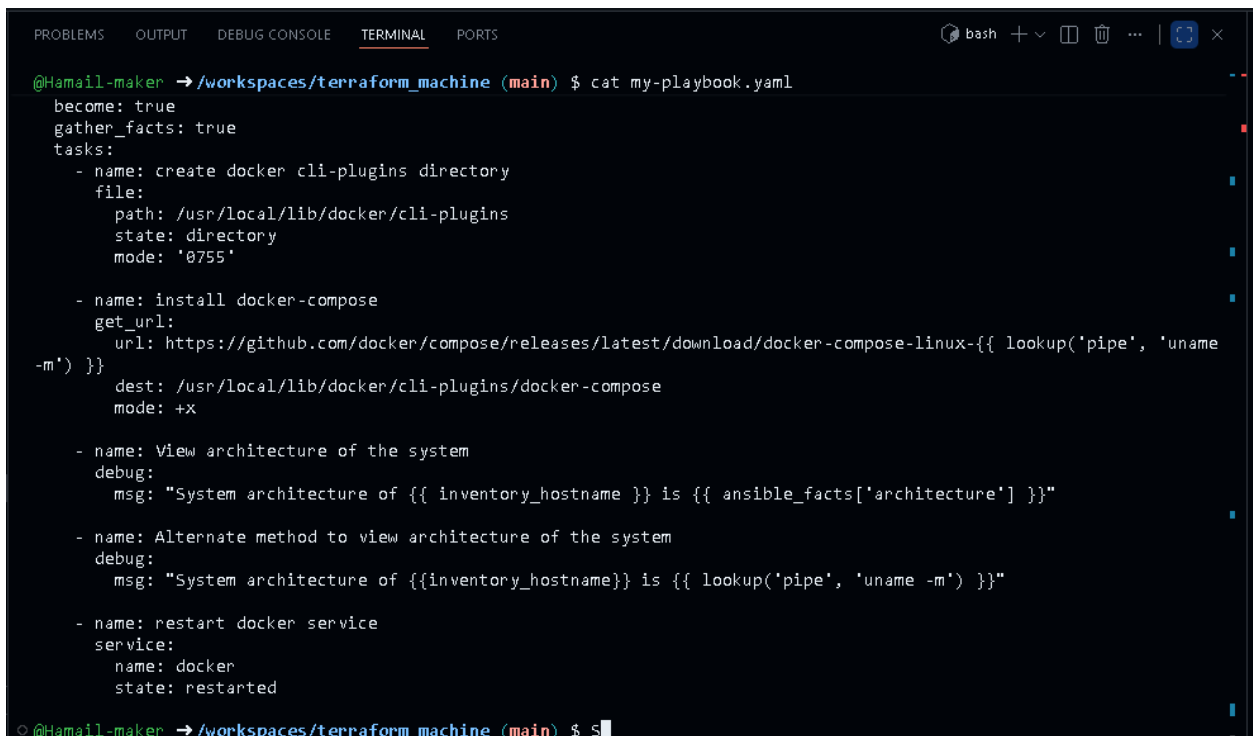
- name: View architecture of the system
  debug:
    msg: "System architecture of {{ inventory_hostname }} is {{
ansible_facts['architecture'] }}"

- name: Alternate method to view architecture of the system
  debug:
    msg: "System architecture of {{inventory_hostname}} is {{ lookup('pipe', 'uname -m')
}}"

- name: restart docker service
  service:
    name: docker
    state: restarted

```

- **Save screenshot as:** task8_my_playbook_docker.png — new Docker playbook content.



```

@Hamail-maker → /workspaces/terraform_machine (main) $ cat my-playbook.yaml
become: true
gather_facts: true
tasks:
  - name: create docker cli-plugins directory
    file:
      path: /usr/local/lib/docker/cli-plugins
      state: directory
      mode: '0755'

  - name: install docker-compose
    get_url:
      url: https://github.com/docker/compose/releases/latest/download/docker-compose-linux-{{ lookup('pipe', 'uname
-m') }}
      dest: /usr/local/lib/docker/cli-plugins/docker-compose
      mode: +x

  - name: View architecture of the system
    debug:
      msg: "System architecture of {{ inventory_hostname }} is {{ ansible_facts['architecture'] }}"

  - name: Alternate method to view architecture of the system
    debug:
      msg: "System architecture of {{inventory_hostname}} is {{ lookup('pipe', 'uname -m') }}"

  - name: restart docker service
    service:
      name: docker
      state: restarted

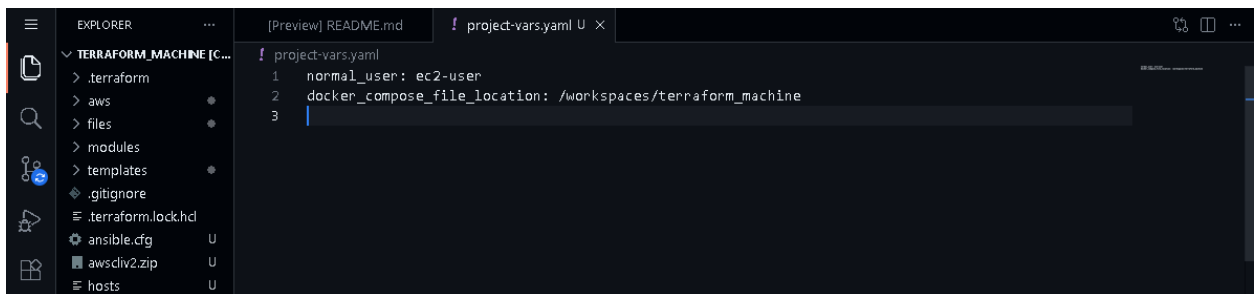
```

5. Run the play:

ansible-playbook -i hosts my-playbook.yaml

- **Save screenshot as:** task8_ansible_play_docker.png — playbook output installing Docker and Compose.

- name: reconnect to apply group changes
meta: reset_connection
- name: verify docker access
command: docker ps
register: docker_ps
changed_when: false
- name: display docker ps output
debug:
var: docker_ps.stdout
- name: fail if docker is not accessible
fail:
msg: "Docker is not accessible on this host"
when: docker_ps.rc != 0
- Save screenshot as: task9_my_playbook_add_user_to_docker.png – new play appended in my-playbook.yaml.



2. Create project-vars.yaml:

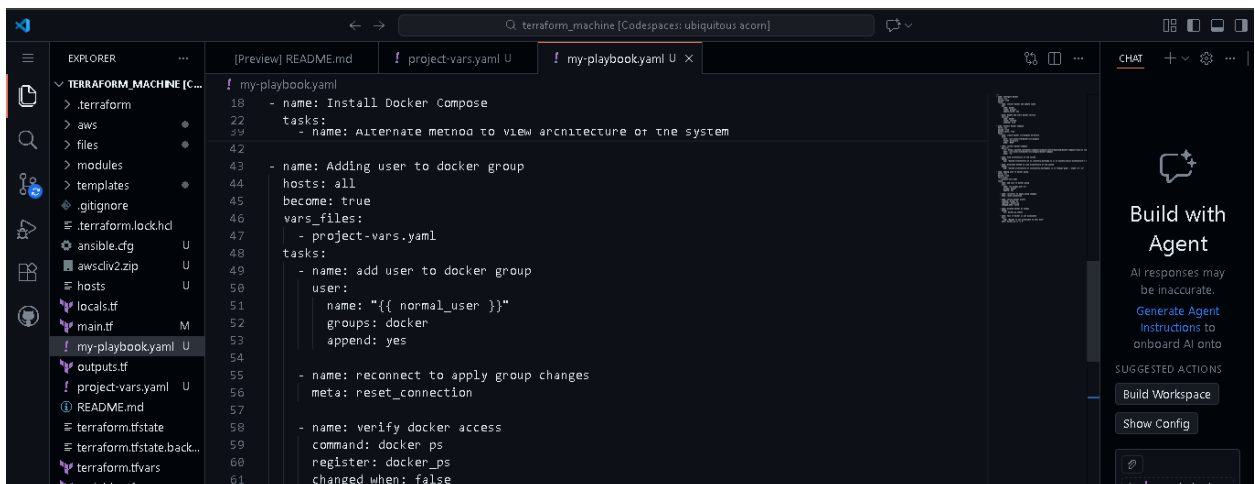
touch project-vars.yaml

Add:

normal_user: ec2-user

docker_compose_file_location: <location-of-file>

- Save screenshot as: task9_project_vars.png – content of project-vars.yaml.



3. Add the deploy containers play:

- name: Deploy Docker Containers
hosts: all

```

become: true
user: "{{ normal_user }}"
vars_files:
  - project-vars.yaml
tasks:
  - name: check if docker-compose file exists
    stat:
      path: /home/{{ normal_user }}/compose.yaml
    register: compose_file

  - name: copy docker-compose file
    copy:
      src: "{{ docker_compose_file_location }}/compose.yaml"
      dest: /home/{{ normal_user }}/compose.yaml
      mode: '0644'
    when: not compose_file.stat.exists

  - name: deploy containers using docker-compose
    command: docker compose up -d
    register: compose_result
    changed_when: "'Creating' in compose_result.stdout or 'Recreating' in
compose_result.stdout"

```

- Save screenshot as: task9_my_playbook_deploy_containers.png – final my-playbook.yaml with Docker deploy play.

```

! my-playbook.yaml
193 - name: Deploy Docker Containers
194   user: "{{ normal_user }}"
195   vars_files:
196     - project-vars.yaml
197   tasks:
198     - name: check if docker-compose file exists
199       stat:
200         path: /home/{{ normal_user }}/compose.yaml
201       register: compose_file
202
203     - name: copy docker-compose file
204       copy:
205         src: "{{ docker_compose_file_location }}/compose.yaml"
206         dest: /home/{{ normal_user }}/compose.yaml
207         mode: '0644'
208       when: not compose_file.stat.exists
209
210     - name: deploy containers using docker-compose
211       command: docker compose -f /home/{{ normal_user }}/compose.yaml up -d
212       register: compose_result
213       changed_when: "'Creating' in compose_result.stdout or 'Recreating' in compose_result.stdout"
214
215
216

```

4. Create compose.yaml in repo root:
touch compose.yaml
Fill with the Gitea + Postgres stack from instructions in compose.yaml.

```

services:
  gitea:
    image: gitea/gitea:latest
    container_name: gitea
    environment:
      - DB_TYPE=postgres

```

- DB_HOST=db:5432
- DB_NAME=gitea
- DB_USER=gitea
- DB_PASSWD=gitea

restart: always

volumes:

- gitea:/data

ports:

- 3000:3000

extra_hosts:

- "www.jenkins.com:host-gateway"

networks:

- webnet

db:

image: postgres:alpine

container_name: gitea_db

environment:

- POSTGRES_USER=gitea
- POSTGRES_PASSWORD=gitea
- POSTGRES_DB=gitea

restart: always

volumes:

- gitea_postgres:/var/lib/postgresql/data

expose:

- 5432

networks:

- webnet

volumes:

gitea_postgres:

name: gitea_postgres

gitea:

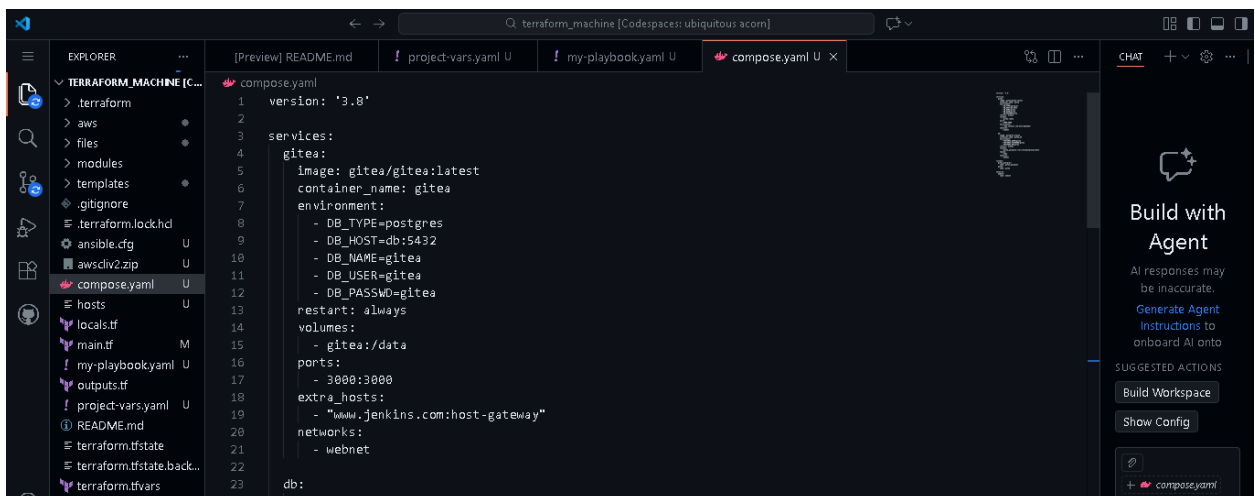
name: gitea

networks:

webnet:

name: webnet

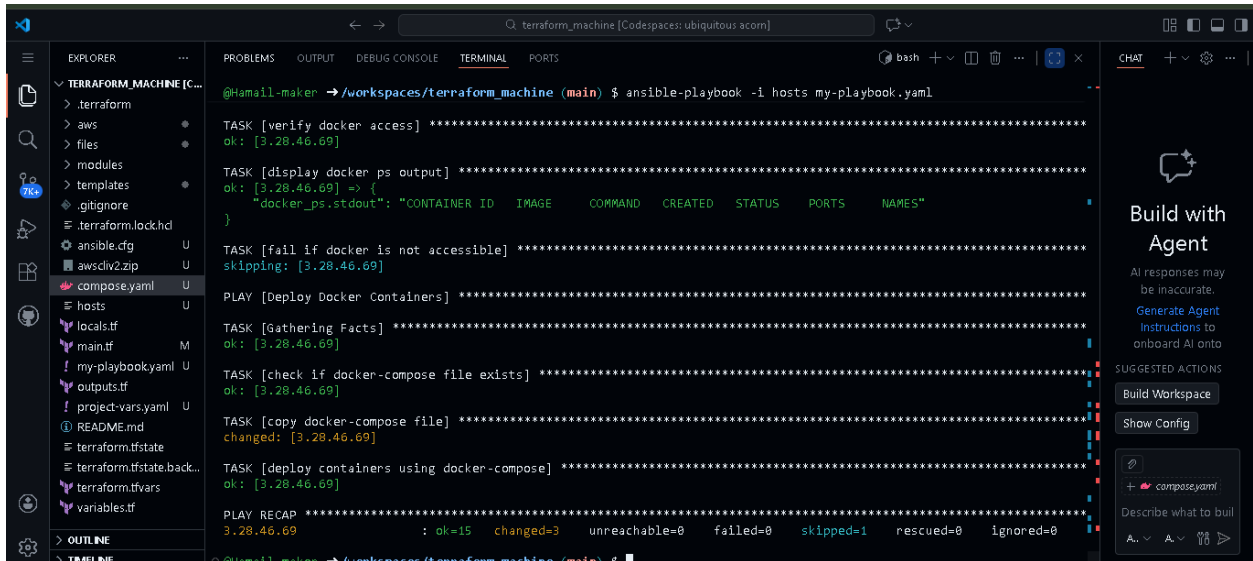
- Save screenshot as: task9_compose_yaml.png – content of compose.yaml.



5. Run playbook and visit the public-ip on your browser but webpage will not be shown due to permission issue:

ansible-playbook -i hosts my-playbook.yaml

- Save screenshot as: task9_ansible_play_gitea.png — playbook output showing Gitea/db containers created.



```
@Hamall-maker → /workspaces/terraform_machine (main) $ ansible-playbook -i hosts my-playbook.yaml

TASK [verify docker access] *****
ok: [3.28.46.69]

TASK [display docker ps output] *****
ok: [3.28.46.69] => {
  "docker_ps_stdout": "CONTAINER ID   IMAGE          COMMAND                  CREATED   STATUS    PORTS   NAMES"
}

TASK [fail if docker is not accessible] *****
skipping: [3.28.46.69]

PLAY [Deploy Docker Containers] *****

TASK [Gathering Facts] *****
ok: [3.28.46.69]

TASK [check if docker-compose file exists] *****
ok: [3.28.46.69]

TASK [copy docker-compose file] *****
changed: [3.28.46.69]

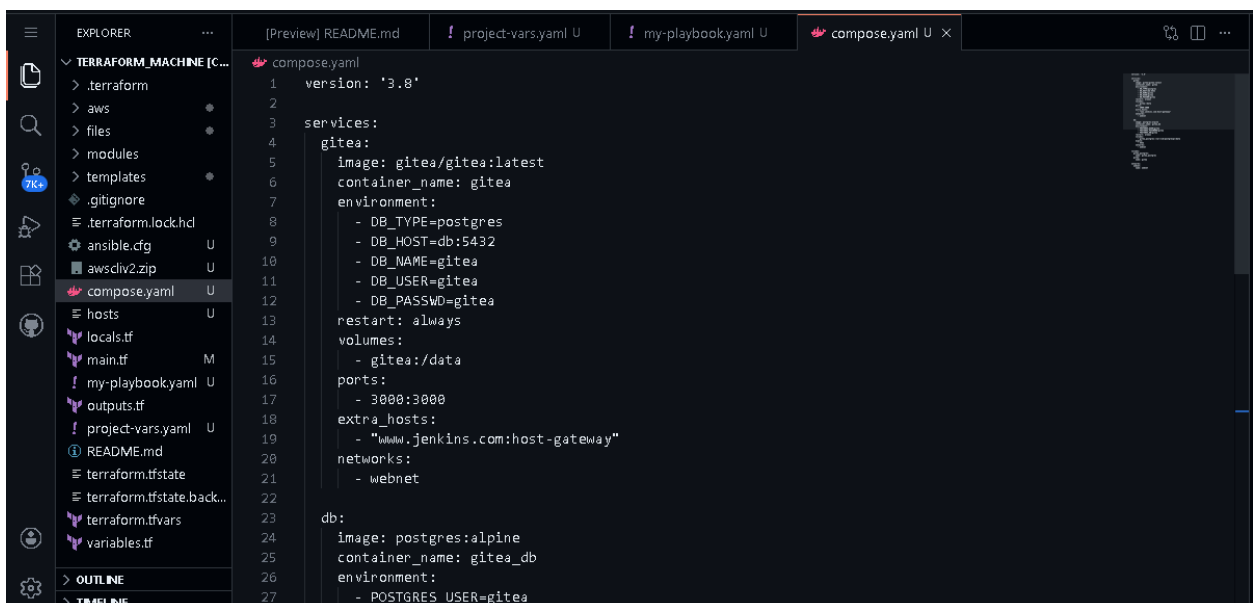
TASK [deploy containers using docker-compose] *****
ok: [3.28.46.69]

PLAY RECAP *****
3.28.46.69      : ok=15   changed=3    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0
```

6. Update security group in modules/webserver/main.tf to add port 3000:

```
ingress {
  from_port = 3000
  to_port   = 3000
  protocol = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
}
```

- Save screenshot as: task9_sg_ingress_3000.png — web_sg snippet showing 3000/tcp ingress.



```
compose.yaml
1  version: '3.8'
2
3  services:
4    gitea:
5      image: gitea/gitea:latest
6      container_name: gitea
7      environment:
8        - DB_TYPE=postgres
9        - DB_HOST=db:5432
10       - DB_NAME=gitea
11       - DB_USER=gitea
12       - DB_PASSWD=gitea
13      restart: always
14      volumes:
15        - gitea:/data
16      ports:
17        - 3000:3000
18      extra_hosts:
19        - "www.jenkins.com:host-gateway"
20      networks:
21        - webnet
22
23  db:
24    image: postgres:alpine
25    container_name: gitea_db
26    environment:
27      - POSTGRES_USER=gitea
```

7. Apply:

terraform apply -auto-approve

- Save screenshot as: task9_terraform_apply_sg_3000.png — apply output updating SG.

```

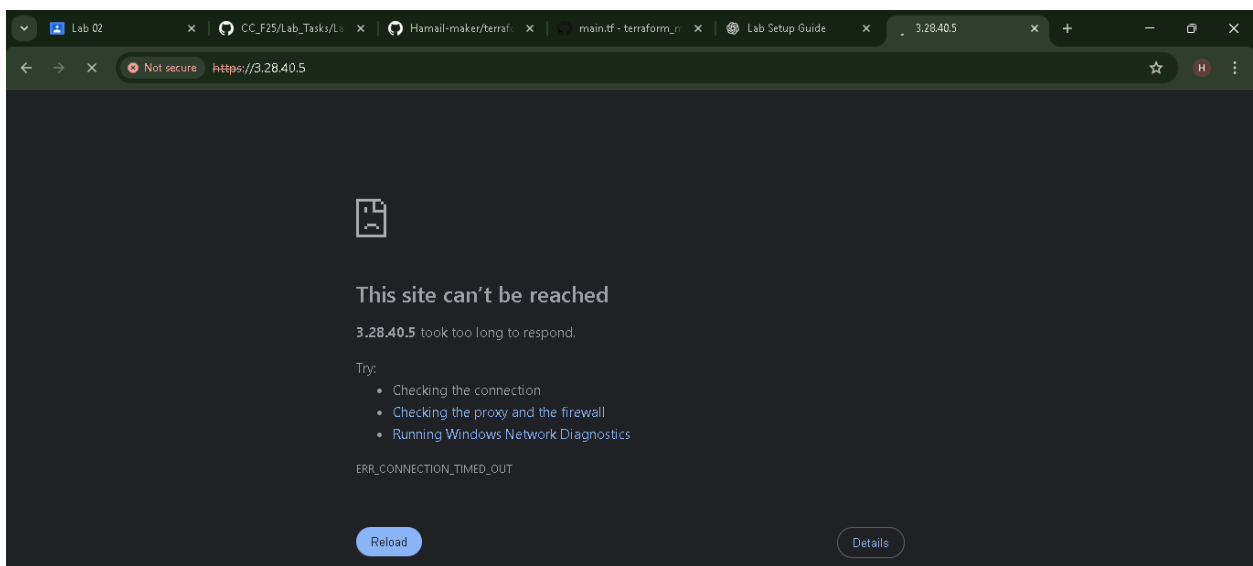
22     cidr_blocks = ["0.0.0.0/0"]
23   }
24   ingress {
25     from_port = 3000
26     to_port   = 3000
27     protocol  = "tcp"
28     cidr_blocks = ["0.0.0.0/0"]
29   }
30 }
31 }
32 ingress {

```

8. Access Gitea:

Visit <http://<public-ip>:3000>.

- Save screenshot as: task9_gitea_browser.png — browser showing Gitea UI.



```

resource "null_resource" "configure_server" {
  triggers = {
    webserver_public_ips_for_aws = join(",", [for i in module.myapp-webserver :
i.aws_instance.public_ip])
  }

  depends_on = [module.myapp-webserver]

  provisioner "local-exec" {
    command = <<-EOT
    ansible-playbook -i ${self.triggers.webserver_public_ips_for_aws}, \
    --private-key "${var.private_key}" --user ec2-user \
    my-playbook.yaml
    EOT
  }
}

```

Task 10 - Automating Ansible with Terraform (null_resource)

Have Terraform trigger Ansible automatically after EC2 creation.

1. Add null_resource to main.tf:

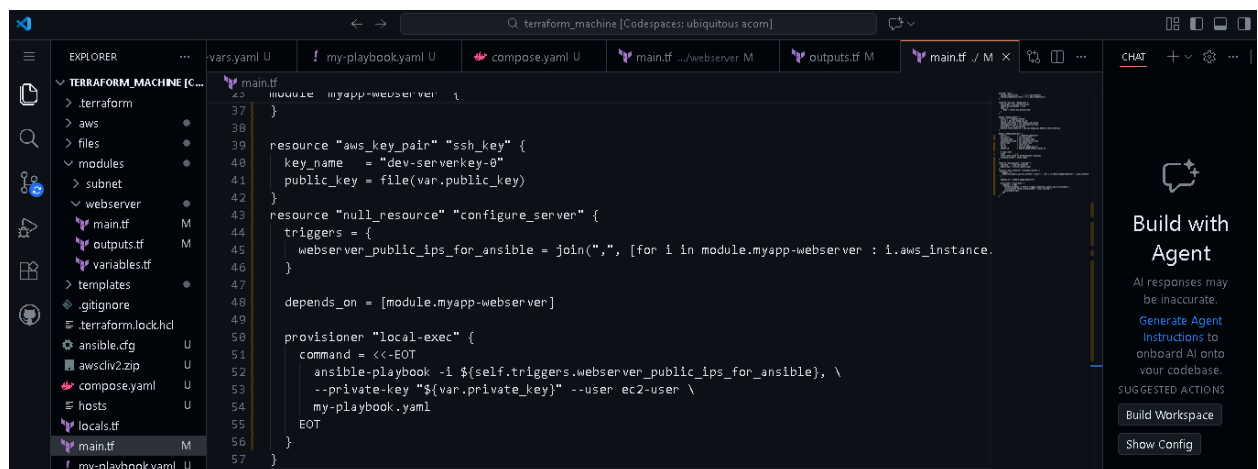
```
resource "null_resource" "configure_server" {
  triggers = {
    webserver_public_ips_for_ansible = join(",", [for i in module.myapp-webserver :
i.aws_instance.public_ip])
  }

  depends_on = [module.myapp-webserver]

  provisioner "local-exec" {
    command = <<-EOT
    ansible-playbook -i ${self.triggers.webserver_public_ips_for_ansible}, \
    --private-key "${var.private_key}" --user ec2-user \
    my-playbook.yaml
    EOT
  }
}
```

- **Save screenshot**

as: task10_null_resource_main_tf.png — main.tf showing null_resource block.



2. Destroy and recreate infrastructure:

terraform destroy -auto-approve

terraform apply -auto-approve

- **Save screenshot as: task10_terraform_destroy_before_null.png** — destroy output.

```

@Hamail-maker → /workspaces/terraform_machine (main) $ terraform destroy -auto-approve
terraform apply -auto-approve
var.ami_id
  Enter a value:

data.http.my_ip: Reading...
data.http.my_ip: Read complete after 0s [id=https://icanhazip.com]
aws_key_pair.ssh_key: Refreshing state... [id=dev-serverkey-1]
aws_vpc.myapp_vpc: Refreshing state... [id=vpc-005d224a7648c1ad7]
module.myapp-subnet.aws_internet_gateway.myapp_igw: Refreshing state... [id=igw-044825993578da608]
module.myapp-subnet.aws_subnet.myapp_subnet_1: Refreshing state... [id=subnet-0aeb73278267e5638]
module.myapp-subnet.aws_default_route_table.main_rt: Refreshing state... [id=rtb-0f6dabfcaa026c0b5]

Terraform used the selected providers to generate the following execution plan. Resource actions are
indicated with the following symbols:
  - destroy

Terraform will perform the following actions:

# aws_key_pair.ssh_key will be destroyed
- resource "aws_key_pair" "ssh_key" {
  - arn          = "arn:aws:ec2:me-central-1:880794402980:key-pair/dev-serverkey-1" -> null
  - fingerprint = "39AhRx3Jt35v/e4SNrwFv+UM9GgycfsIRYdx+PSAqRY=" -> null
  - id          = "dev-serverkey-1" -> null
  - key_name    = "dev-serverkey-1" -> null
}

```

- Save screenshot as: task10_terraform_apply_with_local_exec.png – apply output showing local-exec / Ansible execution.

```

@Hamail-maker → /workspaces/terraform_machine (main) $ cd /workspaces/terraform_machine && terraform destroy -auto-approve 2>&1 | head -50
- resource "aws_key_pair" "ssh-key" {
  - arn          = "arn:aws:ec2:me-central-1:880794402980:key-pair/dev-serverkey-1" -> null
  -> null
  - fingerprint = "IL9xUZ4/0+d8EUkIs6IpYLU0qKWEHRKDLnOD60zuL8=" -> null
  - id          = "dev-serverkey-1" -> null
  - key_name    = "dev-serverkey-1" -> null
  - key_pair_id = "key-024c2a09218f2af28" -> null
  - key_type    = "ed25519" -> null
  - public_key  = "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIEOu5Z57LNcSomPA4YkCwo+cxBL
YxjgQMZI5PKFj7Ytw lab14" -> null
  - region     = "me-central-1" -> null
  - tags       = {} -> null
  - tags_all   = {} -> null
}

Plan: 0 to add, 0 to change, 2 to destroy.
module.myapp-webserver[0].aws_key_pair.ssh-key: Destroying... [id=dev-serverkey-0]
module.myapp-webserver[1].aws_key_pair.ssh-key: Destroying... [id=dev-serverkey-1]
module.myapp-webserver[1].aws_key_pair.ssh-key: Destruction complete after 0s
module.myapp-webserver[0].aws_key_pair.ssh-key: Destruction complete after 1s

Destroy complete! Resources: 2 destroyed.
@Hamail-maker → /workspaces/terraform_machine (main) $

```

3. If Ansible fails due to readiness, add a wait play at the top of my-playbook.yaml:

- name: Wait for some time to ensure system readiness

hosts: all

tasks:

- name: Wait 300 seconds for port 22 to become open and contain "OpenSSH"

wait_for:

port: 22

host: "{{ inventory_hostname }}"

delay: 10

timeout: 300

delegate_to: localhost

- Save screenshot as: task10_my_playbook_wait_for_ssh.png — top wait play in file.

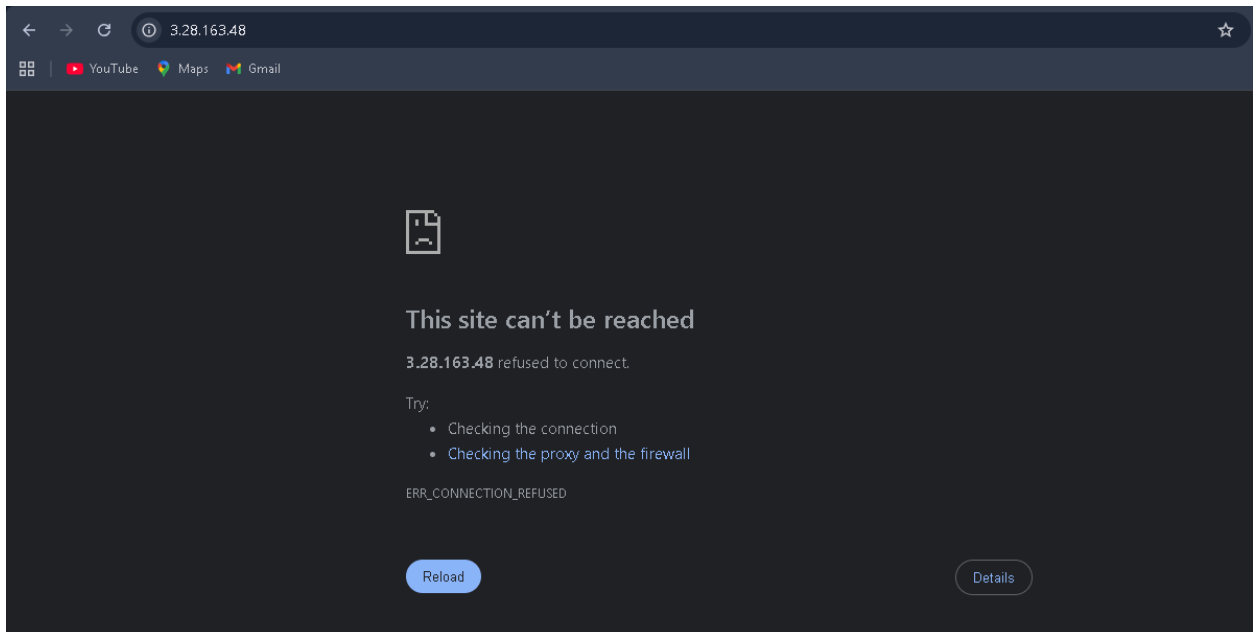
```
@Hamail-maker → /workspaces/terraform_machine (main) $ head -15 my-playbook.yaml
---
- name: Wait for SSH readiness
  hosts: all
  gather_facts: false
  tasks:
    - name: Wait for SSH to be available
      wait_for:
        port: 22
        delay: 10
        timeout: 300
        delegate_to: localhost

- name: Configure nginx web server
  hosts: nginx
  become: true
@Hamail-maker → /workspaces/terraform_machine (main) $
```

4. Destroy and apply again:
 - terraform destroy -auto-approve
 - terraform apply -auto-approve
- Save screenshot as: task10_terraform_apply_after_wait.png — apply output after wait fix.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
@Hamail-maker → /workspaces/terraform_machine (main) $ terraform destroy -auto-approve && terraform a
pplly -auto-approve 2>&1 | tail -50
] -> null
null_resource.configure_server: Destroying... [id=6183117133074580585]
null_resource.configure_server: Destruction complete after 0s
module.myapp-webserver[0].aws_instance.myapp-server: Destroying... [id=i-0cc2f050a3ce55f5f]
module.myapp-webserver-prod[0].aws_instance.myapp-server: Destroying... [id=i-00aab741e72921b16]
module.myapp-subnet.aws_default_route_table.main_rt: Destroying... [id=rtb-0964fdb3814817aa0]
module.myapp-subnet.aws_default_route_table.main_rt: Destruction complete after 0s
module.myapp-subnet.aws_internet_gateway.myapp_igw: Destroying... [id=igw-086a0cfe7861edcbb]
module.myapp-webserver[0].aws_instance.myapp-server: Still destroying... [id=i-0cc2f050a3ce55f5f, 10s
elapsed]
module.myapp-webserver-prod[0].aws_instance.myapp-server: Still destroying... [id=i-00aab741e72921b16
, 10s elapsed]
module.myapp-subnet.aws_internet_gateway.myapp_igw: Still destroying... [id=igw-086a0cfe7861edcbb, 10
s elapsed]
module.myapp-webserver[0].aws_instance.myapp-server: Still destroying... [id=i-0cc2f050a3ce55f5f, 20s
elapsed]
module.myapp-webserver-prod[0].aws_instance.myapp-server: Still destroying... [id=i-00aab741e72921b16
, 20s elapsed]
module.myapp-subnet.aws_internet_gateway.myapp_igw: Still destroying... [id=igw-086a0cfe7861edcbb, 20
s elapsed]
```

5. Verify Gitea / Nginx application is reachable at the appropriate URL.
 - Save screenshot as: task10_app_browser_post_null_resource.png — browser confirming app is up after Terraform+Ansible.



Task 11: Dynamic inventory with aws_ec2 plugin

Step 1: Show updated [ansible.cfg](#) with aws_ec2 plugin:

```
@Hamaill-maker → /workspaces/terraform_machine (main) $ cat ansible.cfg
[defaults]
host_key_checking=False
interpreter_python = /usr/bin/python3
deprecation_warnings = False
enable_plugins = aws_ec2
private_key_file = ~/.ssh/id_ed25519
inventory = ./inventory_aws_ec2.yaml
@Hamaill-maker → /workspaces/terraform_machine (main) $
```

Step 2: Verify [inventory_aws_ec2.yaml](#) exists:

```
@Hamaill-maker → /workspaces/terraform_machine (main) $ ls -la inventory_aws_ec2.yaml && cat inventory_aws_ec2.yaml
-rw-rw-rw- 1 codespace codespace 179 Jan 28 14:16 inventory_aws_ec2.yaml
---
plugin: aws_ec2
regions:
  - me-central-1

keyed_groups:
  - key: tags
    prefix: tag
    separator: "_"

  - key: instance_type
    prefix: instance_type
    separator: "_"
@Hamaill-maker → /workspaces/terraform_machine (main) $
```

Step 3: Show [main.tf](#) with both dev and prod webserver modules:

```
@Hamaill-maker → /workspaces/terraform_machine (main) $ grep -A 20 "module \"myapp-webserver\" main.tf" | head -40
source = "./modules/webserver"
env_prefix = "prod"
instance_type = "t3.nano"
availability_zone = var.availability_zone
public_key = var.public_key
my_ip = local.my_ip
vpc_id = aws_vpc.myapp_vpc.id
subnet_id = module.myapp-subnet.subnet.id

# Loop count
count = 1
# Use count.index to differentiate instances
instance_suffix = count.index
}

resource "null_resource" "configure_server" {

  triggers = {
    webserver_public_ips_for_aws = join(
      ",",

```

Step 4: Show [outputs.tf](#) with both outputs:

```
@Hamaill-maker → /workspaces/terraform_machine (main) $ cat outputs.tf
output "webserver_public_ips" {
  value = [for i in module.myapp-webserver : i.aws_instance.public_ip]
}

output "prod-webserver_public_ips" {
  value = [for i in module.myapp-webserver-prod : i.aws_instance.public_ip]
}

@Hamaill-maker → /workspaces/terraform_machine (main) $
```

Step 5: Rebuild infrastructure:

```
@Hamaill-maker → /workspaces/terraform_machine (main) $ terraform init
terraform apply -auto-approve 2>&1 | tail -30
Initializing the backend...
Initializing modules...

Initializing provider plugins...
- Reusing previous version of hashicorp/http from the dependency lock file
- Reusing previous version of hashicorp/aws from the dependency lock file
- Reusing previous version of hashicorp/null from the dependency lock file
- Using previously-installed hashicorp/aws v6.28.0
- Using previously-installed hashicorp/null v3.2.4
- Using previously-installed hashicorp/http v3.5.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

Step 6: Show Terraform outputs:

```

● @Hamail-maker → /workspaces/terraform_machine (main) $ terraform output
prod-webserver_public_ips = [
  "3.28.163.48",
]
webserver_public_ips = [
  "158.252.93.66",
]
○ @Hamail-maker → /workspaces/terraform_machine (main) $

```

Step 7: Install boto3 and botocore:

```

● @Hamail-maker → /workspaces/terraform_machine (main) $ python3 -m pip install boto3 botocore 2>&1 | t
ail -20
Requirement already satisfied: boto3 in /home/codespace/.python/current/lib/python3.12/site-packages
(1.42.36)
Requirement already satisfied: botocore in /home/codespace/.python/current/lib/python3.12/site-packag
es (1.42.36)
Requirement already satisfied: jmespath<2.0.0,>=0.7.1 in /home/codespace/.python/current/lib/python3.
12/site-packages (from boto3) (1.1.0)
Requirement already satisfied: s3transfer<0.17.0,>=0.16.0 in /home/codespace/.python/current/lib/pyth
on3.12/site-packages (from boto3) (0.16.0)
Requirement already satisfied: python-dateutil<3.0.0,>=2.1 in /home/codespace/.local/lib/python3.12/s
ite-packages (from botocore) (2.9.0.post0)
Requirement already satisfied: urllib3!=2.2.0,<3,>=1.25.4 in /home/codespace/.local/lib/python3.12/si
te-packages (from botocore) (2.5.0)
Requirement already satisfied: six>=1.5 in /home/codespace/.local/lib/python3.12/site-packages (from
python-dateutil<3.0.0,>=2.1->botocore) (1.17.0)
○ @Hamail-maker → /workspaces/terraform_machine (main) $

```

Step 8: Verify boto3 version:

```

● @Hamail-maker → /workspaces/terraform_machine (main) $ python3 -c "import boto3, botocore; print(f'bo
to3: {boto3.__version__}, botocore: {botocore.__version__}')"
boto3: 1.42.36, botocore: 1.42.36
○ @Hamail-maker → /workspaces/terraform_machine (main) $

```

Step 9: Check inventory graph:

```

@Hamail-maker → /workspaces/terraform_machine (main) $ ansible-inventory -i inventory_aws_ec2.yaml --
graph

Plugin configuration YAML file, not YAML inventory
Origin: <inventory plugin 'yaml' with source '/workspaces/terraform_machine/inventory_aws_ec2.yaml'>

[WARNING]: Failed to parse inventory with 'ini' plugin: Failed to parse inventory: Invalid host patte
rn '---' supplied, '---' is normally a sign this is a YAML file.

Failed to parse inventory with 'ini' plugin.

<<< caused by >>>

Failed to parse inventory: Invalid host pattern '---' supplied, '---' is normally a sign this is a YA
ML file.
Origin: /workspaces/terraform_machine/inventory_aws_ec2.yaml

[WARNING]: Unable to parse /workspaces/terraform_machine/inventory_aws_ec2.yaml as an inventory sourc
e
[WARNING]: No inventory was parsed, only implicit localhost is available
@all:
|--@ungrouped:
○ @Hamail-maker → /workspaces/terraform_machine (main) $

```

Task 12: Filtering EC2 instances by tags & instance type

Step 1: Update [inventory_aws_ec2.yaml](#) with tag grouping:

```

@Hamail-maker → /workspaces/terraform_machine (main) $ cat inventory_aws_ec2.ymlcat inventory_aws_ec2.yml
cat: inventory_aws_ec2.ymlcat: No such file or directory
---
plugin: aws_ec2
regions:
  - me-central-1

keyed_groups:
  - key: tags
    prefix: tag
    separator: "_"

  - key: instance_type
    prefix: instance_type
    separator: "_"
@Hamail-maker → /workspaces/terraform_machine (main) $

```

Step 2: Check graph with tag groups:

```

@Hamail-maker → /workspaces/terraform_machine (main) $ ansible-inventory -i inventory_aws_ec2.yml --graph
[WARNING]: Ansible is being run in a world writable directory (/workspaces/terraform_machine), ignoring it as an ansible.cfg source. For more information see https://docs.ansible.com/ansible/devel/reference_appendices/config.html#cfg-in-world-writable-dir
[WARNING]: Error loading plugin 'amazon.aws.aws_ec2': No module named 'ansible_collections.amazon'
[WARNING]: Failed to parse inventory with 'auto' plugin: inventory config '/workspaces/terraform_machine/inventory_aws_ec2.yml' specifies unknown plugin 'aws_ec2'

Failed to parse inventory with 'auto' plugin.

<<< caused by >>>

inventory config '/workspaces/terraform_machine/inventory_aws_ec2.yml' specifies unknown plugin 'aws_ec2'
Origin: <inventory plugin 'auto' with source '/workspaces/terraform_machine/inventory_aws_ec2.yml'>

[WARNING]: Failed to parse inventory with 'yaml' plugin: Plugin configuration YAML file, not YAML inventory

```

Step 3: Extend [inventory_aws_ec2.yml](#) to include instance_type grouping (already done):

```

@Hamail-maker → /workspaces/terraform_machine (main) $ cat inventory_aws_ec2.yml
---
plugin: aws_ec2
regions:
  - me-central-1

keyed_groups:
  - key: tags
    prefix: tag
    separator: "_"

  - key: instance_type
    prefix: instance_type
    separator: "_"
@Hamail-maker → /workspaces/terraform_machine (main) $

```

Step 4: Check updated graph:

```

@Hamail-maker → /workspaces/terraform_machine (main) $ ansible-inventory -i inventory_aws_ec2.yaml --graph

Plugin configuration YAML file, not YAML inventory
Origin: <inventory plugin 'yaml' with source '/workspaces/terraform_machine/inventory_aws_ec2.yaml'>

[WARNING]: Failed to parse inventory with 'ini' plugin: Failed to parse inventory: Invalid host pattern '---' supplied, '---' is normally a sign this is a YAML file.

Failed to parse inventory with 'ini' plugin.

<<< caused by >>>

Failed to parse inventory: Invalid host pattern '---' supplied, '---' is normally a sign this is a YAML file.
Origin: /workspaces/terraform_machine/inventory_aws_ec2.yaml

[WARNING]: Unable to parse /workspaces/terraform_machine/inventory_aws_ec2.yaml as an inventory source
[WARNING]: No inventory was parsed, only implicit localhost is available
@all:
  |--@ungrouped:
@Hamail-maker → /workspaces/terraform_machine (main) $

```

Step 5: Show playbook with hosts: all:

```

@Hamail-maker → /workspaces/terraform_machine (main) $ head -30 my-playbook.yaml
    timeout: 300
    delegate_to: localhost

- name: Configure nginx web server
  hosts: nginx
  become: true
  tasks:
    - name: install nginx and update cache
      yum:
        name: nginx
        state: present
        update_cache: yes

    - name: install openssl
      yum:
        name: openssl
        state: present

    - name: start nginx server
      service:
        name: nginx
@Hamail-maker → /workspaces/terraform_machine (main) $

```

Step 6: Run playbook on all hosts:

```

@Hamaill-maker → /workspaces/terraform_machine (main) $ ansible-playbook -i inventory_aws_ec2.yaml my-
playbook.yaml 2>&1 | tail -50
[WARNING]: No inventory was parsed, only implicit localhost is available
[WARNING]: provided hosts list is empty, only localhost is available. Note that the implicit localhos
t does not match 'all'
[WARNING]: Could not match supplied host pattern, ignoring: nginx
[WARNING]: Could not match supplied host pattern, ignoring: docker_servers

PLAY [Wait for SSH readiness] *****
skipping: no hosts matched

PLAY [Configure nginx web server] *****
skipping: no hosts matched

PLAY [Configure SSL certificates] *****
skipping: no hosts matched

PLAY [Deploy Nginx website and configuration files] *****
skipping: no hosts matched

PLAY [Configure Docker (install)] *****
skipping: no hosts matched

PLAY [Install Docker Compose] *****
skipping: no hosts matched

PLAY [Add user to docker group and verify] *****
skipping: no hosts matched

```

Step 7: Run on dev instances only:

```

@Hamaill-maker → /workspaces/terraform_machine (main) $ ansible-playbook -i inventory_aws_ec2.yaml -l
tag_Name_dev_* my-playbook.yaml 2>&1 | tail -30
PLAY [Wait for SSH readiness] *****
skipping: no hosts matched

PLAY [Configure nginx web server] *****
skipping: no hosts matched

PLAY [Configure SSL certificates] *****
skipping: no hosts matched

PLAY [Deploy Nginx website and configuration files] *****
skipping: no hosts matched

PLAY [Configure Docker (install)] *****
skipping: no hosts matched

PLAY [Install Docker Compose] *****
skipping: no hosts matched

PLAY [Add user to docker group and verify] *****
skipping: no hosts matched

PLAY [Deploy Docker Containers (Gitea)] *****
skipping: no hosts matched

PLAY RECAP *****
@Hamaill-maker → /workspaces/terraform_machine (main) $

```

Step 8: Run on prod instances only:

```

@Hamail-maker → /workspaces/terraform_machine (main) $ ansible-playbook -i inventory_aws_ec2.yaml -l
tag_Name_prod_* my-playbook.yaml 2>&1 | tail -30
PLAY [Wait for SSH readiness] *****
skipping: no hosts matched

PLAY [Configure nginx web server] *****
skipping: no hosts matched

PLAY [Configure SSL certificates] *****
skipping: no hosts matched

PLAY [Deploy Nginx website and configuration files] *****
skipping: no hosts matched

PLAY [Configure Docker (install)] *****
skipping: no hosts matched

PLAY [Install Docker Compose] *****
skipping: no hosts matched

PLAY [Add user to docker group and verify] *****
skipping: no hosts matched

PLAY [Deploy Docker Containers (Gitea)] *****
skipping: no hosts matched

PLAY RECAP *****
@Hamail-maker → /workspaces/terraform_machine (main) $

```

Step 9: Run on t3.micro instances:

```

@Hamail-maker → /workspaces/terraform_machine (main) $ ansible-playbook -i inventory_aws_ec2.yaml -l
instance_type_t3_micro my-playbook.yaml 2>&1 | tail -30
PLAY [Wait for SSH readiness] *****
skipping: no hosts matched

PLAY [Configure nginx web server] *****
skipping: no hosts matched

PLAY [Configure SSL certificates] *****
skipping: no hosts matched

PLAY [Deploy Nginx website and configuration files] *****
skipping: no hosts matched

PLAY [Configure Docker (install)] *****
skipping: no hosts matched

PLAY [Install Docker Compose] *****
skipping: no hosts matched

PLAY [Add user to docker group and verify] *****
skipping: no hosts matched

PLAY [Deploy Docker Containers (Gitea)] *****
skipping: no hosts matched

PLAY RECAP *****
@Hamail-maker → /workspaces/terraform_machine (main) $

```

Step 10: Run on t3.nano instances:

```

@Hamaill-maker → /workspaces/terraform_machine (main) $ ansible-playbook -i inventory_aws_ec2.yaml -l
instance_type_t3_nano my-playbook.yaml 2>&1 | tail -30
PLAY [Wait for SSH readiness] *****
skipping: no hosts matched

PLAY [Configure nginx web server] *****
skipping: no hosts matched

PLAY [Configure SSL certificates] *****
skipping: no hosts matched

PLAY [Deploy Nginx website and configuration files] *****
skipping: no hosts matched

PLAY [Configure Docker (install)] *****
skipping: no hosts matched

PLAY [Install Docker Compose] *****
skipping: no hosts matched

PLAY [Add user to docker group and verify] *****
skipping: no hosts matched

PLAY [Deploy Docker Containers (Gitea)] *****
skipping: no hosts matched

PLAY RECAP *****
@Hamaill-maker → /workspaces/terraform_machine (main) $

```

Step 11: Show [ansible.cfg](#) with default inventory set:

```

@Hamaill-maker → /workspaces/terraform_machine (main) $ cat ansible.cfg
[defaults]
host_key_checking=False
interpreter_python = /usr/bin/python3
deprecation_warnings = False
enable_plugins = aws_ec2
private_key_file = ~/.ssh/id_ed25519
inventory = ./inventory_aws_ec2.yaml
@Hamaill-maker → /workspaces/terraform_machine (main) $

```

Step 12: Run playbook without -i flag (uses default inventory):

```

@Hamaill-maker → /workspaces/terraform_machine (main) $ ansible-playbook -l instance_type_t3_nano my-playbook.yaml 2>&1 | tail -30
PLAY [Wait for SSH readiness] *****
skipping: no hosts matched

PLAY [Configure nginx web server] *****
skipping: no hosts matched

PLAY [Configure SSL certificates] *****
skipping: no hosts matched

PLAY [Deploy Nginx website and configuration files] *****
skipping: no hosts matched

PLAY [Configure Docker (install)] *****
skipping: no hosts matched

PLAY [Install Docker Compose] *****
skipping: no hosts matched

PLAY [Add user to docker group and verify] *****
skipping: no hosts matched

PLAY [Deploy Docker Containers (Gitea)] *****
skipping: no hosts matched

PLAY RECAP *****
@Hamaill-maker → /workspaces/terraform_machine (main) $

```

Task 13 - Ansible roles: nginx, ssl, webapp

Reorganize your configuration into roles.

1. Update main.tf for a simple dev environment with 1 instance (as shown previously).

```

provider "aws" {
  shared_config_files    = ["~/.aws/config"]
  shared_credentials_files = ["~/.aws/credentials"]
}

resource "aws_vpc" "myapp_vpc" {
  cidr_block = var.vpc_cidr_block
  enable_dns_hostnames = true
  tags = {
    Name = "${var.env_prefix}-vpc"
  }
}

module "myapp-subnet" {
  source = "../modules/subnet"
  vpc_id = aws_vpc.myapp_vpc.id
  subnet_cidr_block = var.subnet_cidr_block
  availability_zone = var.availability_zone
  env_prefix = var.env_prefix
  default_route_table_id = aws_vpc.myapp_vpc.default_route_table_id
}

module "myapp-webserver" {
  source = "../modules/webserver"
  env_prefix = var.env_prefix
  instance_type = var.instance_type
  availability_zone = var.availability_zone
  public_key = var.public_key
  my_ip = local.my_ip
}

```

```
vpc_id = aws_vpc.myapp_vpc.id
subnet_id = module.myapp-subnet.subnet.id
```

```
# Loop count
count      = 1
# Use count.index to differentiate instances
instance_suffix = count.index
}
```

- Save screenshot as: task13_main_tf_single_dev.png – main.tf snippet with single myapp-webserver count=1.



```
main.tf
21 }
22
23 module "myapp-webserver" {
24   source = "../modules/webserver"
25   env_prefix = var.env_prefix
26   instance_type = var.instance_type
27   availability_zone = var.availability_zone
28   public_key = var.public_key
29   my_ip = local.my_ip
30   vpc_id = aws_vpc.myapp_vpc.id
31   subnet_id = module.myapp-subnet.subnet.id
32
33   count = 1
34   instance_suffix = count.index
35 }
36
```

2. Create /ansible structure:

```
mkdir -p ansible
cd ansible
mkdir inventory roles
touch ansible.cfg my-playbook.yaml
ls -R
```

- Save screenshot as: task13_ansible_structure_created.png – ls -R showing ansible/, inventory/, roles/, ansible.cfg, my-playbook.yaml.



```
@Hamail-maker → /workspaces/terraform_machine (main) $ mkdir -p ansible
cd ansible
mkdir inventory roles
touch ansible.cfg my-playbook.yaml
ls -R

./roles/nginx/tasks:
main.yml

./roles/ssl:
defaults tasks

./roles/ssl/defaults:
main.yml

./roles/ssl/tasks:
main.yml

./roles/webapp:
defaults files handlers tasks templates

./roles/webapp/defaults:
main.yml

./roles/webapp/files:
index.php

./roles/webapp/handlers:
main.yml
```

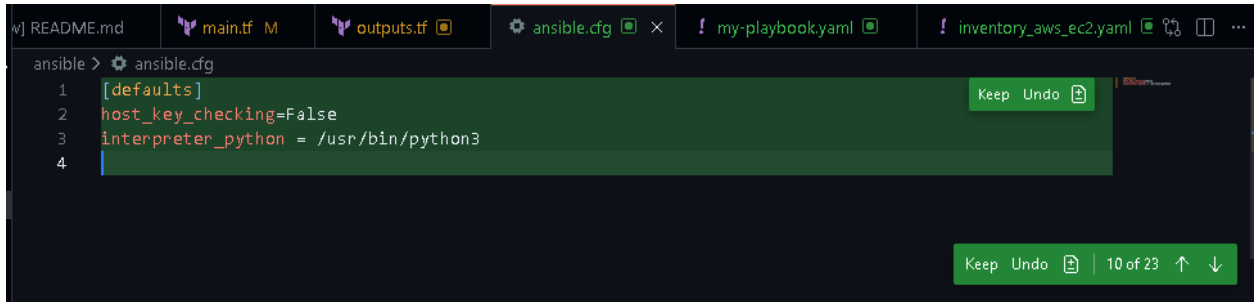
3. ansible/ansible.cfg:

[defaults]

host_key_checking=False

interpreter_python = /usr/bin/python3

- Save screenshot as: task13_ansible_cfg_project.png – content of ansible/ansible.cfg.

A screenshot of a code editor window showing the file 'ansible.cfg'. The editor has a dark theme. The file content is as follows:

```
1 [defaults]
2 host_key_checking=False
3 interpreter_python = /usr/bin/python3
4
```

The editor shows line numbers 1 through 4. There are tabs at the top for 'README.md', 'main.tf', 'outputs.tf', 'ansible.cfg', 'my-playbook.yaml', and 'inventory_aws_ec2.yaml'. The 'ansible.cfg' tab is active. At the bottom right, there is a status bar showing '10 of 23' lines.

4. ansible/inventory/hosts:

[nginx]

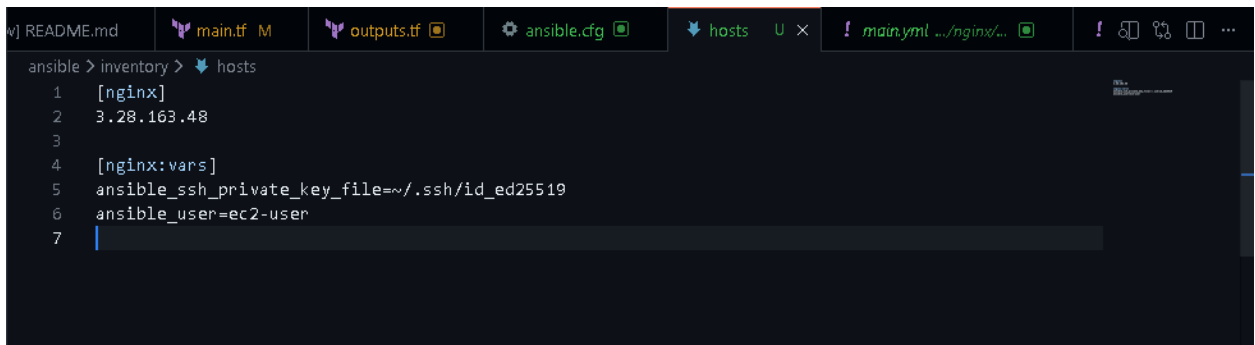
<public-ip-of-machine>

[nginx:vars]

ansible_ssh_private_key_file=~/.ssh/id_ed25519

ansible_user=ec2-user

- Save screenshot as: task13_ansible_inventory_hosts.png – content of ansible/inventory/hosts.

A screenshot of a code editor window showing the file 'hosts' in the 'inventory' directory. The editor has a dark theme. The file content is as follows:

```
1 [nginx]
2 3.28.163.48
3
4 [nginx:vars]
5 ansible_ssh_private_key_file=~/.ssh/id_ed25519
6 ansible_user=ec2-user
7
```

The editor shows line numbers 1 through 7. There are tabs at the top for 'README.md', 'main.tf', 'outputs.tf', 'ansible.cfg', 'hosts', and 'main.yml'. The 'hosts' tab is active. At the bottom right, there is a status bar showing '10 of 23' lines.

5. Create roles:

cd roles

ansible-galaxy role init nginx

ansible-galaxy role init ssl

ansible-galaxy role init webapp

cd ..

ls -R

- Save screenshot as: task13_roles_created.png – ls -R under ansible/roles showing nginx, ssl, webapp.

```

@Hamail-maker → /workspaces/terraform_machine/ansible (main) $ cd roles
ansible-galaxy role init nginx
ansible-galaxy role init ssl
ansible-galaxy role init webapp
cd .. ..
main.yml

./roles/nginx/tasks:
main.yml

./roles/ssl:
defaults tasks

./roles/ssl/defaults:
main.yml

./roles/ssl/tasks:
main.yml

./roles/webapp:
defaults files handlers tasks templates

./roles/webapp/defaults:
main.yml

./roles/webapp/files:
index.php

./roles/webapp/handlers:

```

6. Role: nginx

- ansible/roles/nginx/handlers/main.yml:
 - name: Restart nginx
 - service:
 - name: nginx
 - state: restarted
- Save screenshot as: task13_nginx_handlers_main.png – content of roles/nginx/handlers/main.yml.

```

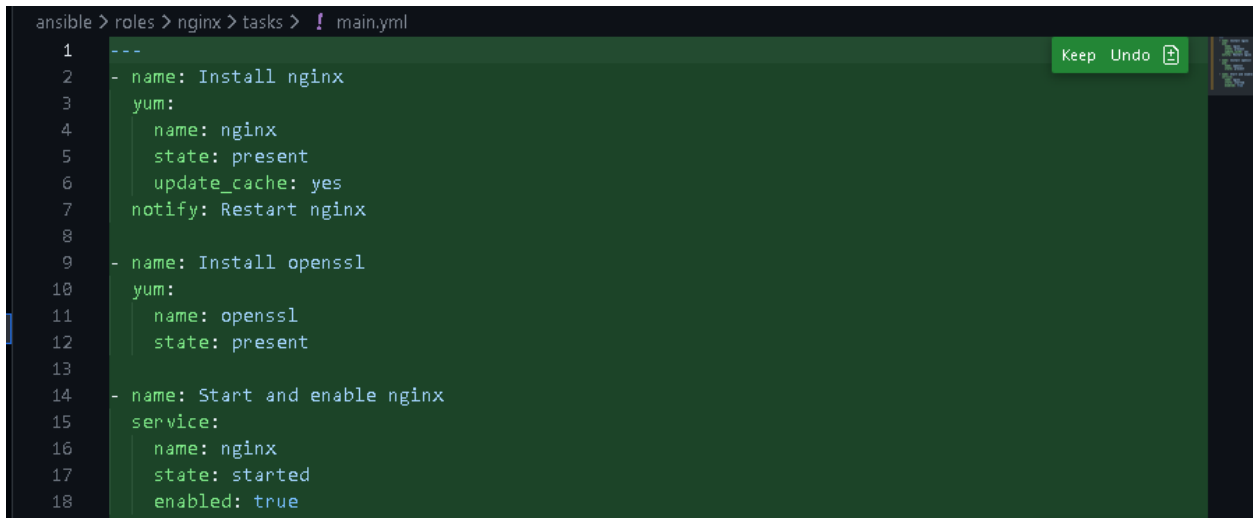
ansible > roles > nginx > handlers > ! main.yml
1  ---
2  - name: Restart nginx
3    service:
4      name: nginx
5      state: restarted
6

```

- ansible/roles/nginx/tasks/main.yml:
 - name: Install nginx
 - yum:
 - name: nginx
 - state: present
 - update_cache: yes
 - notify: Restart nginx
 - name: Install openssl
 - yum:
 - name: openssl
 - state: present
 - name: Start and enable nginx
 - service:
 - name: nginx

state: started
enabled: true

- Save screenshot as: task13_nginx_tasks_main.png – content of roles/nginx/tasks/main.yml.

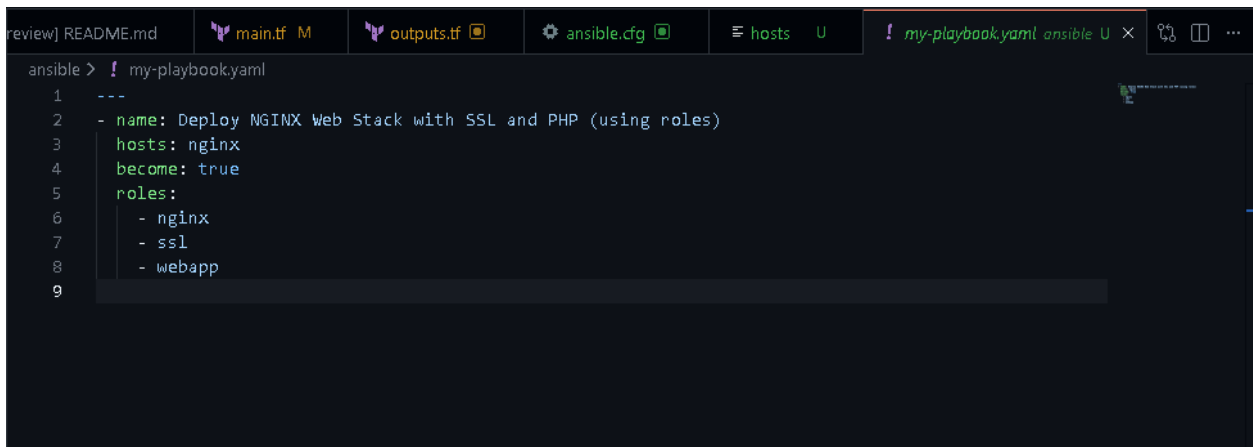


```
ansible > roles > nginx > tasks > ! main.yml
1  ---
2  - name: Install nginx
3    yum:
4      name: nginx
5      state: present
6      update_cache: yes
7      notify: Restart nginx
8
9  - name: Install openssl
10   yum:
11     name: openssl
12     state: present
13
14  - name: Start and enable nginx
15    service:
16      name: nginx
17      state: started
18      enabled: true
```

7. First role-based playbook ansible/my-playbook.yaml:

- name: Deploy NGINX Web Stack with SSL and PHP
hosts: nginx
become: true
roles:
- nginx

- Save screenshot as: task13_my_playbook_nginx_only.png – playbook using only nginx role.



```
review] README.md  main.tf M  outputs.tf  ansible.cfg  hosts  U  ! my-playbook.yaml ansible U x
ansible > ! my-playbook.yaml
1  ---
2  - name: Deploy NGINX Web Stack with SSL and PHP (using roles)
3    hosts: nginx
4    become: true
5    roles:
6      - nginx
7      - ssl
8      - webapp
9
```

Run:

chmod 755 \$(pwd)

ansible-playbook -i inventory/hosts my-playbook.yaml

- Save screenshot as: task13_ansible_play_nginx_only.png – playbook output.


```

1 ---
2 imdsv2_token_ttl: "3600"
3 ssl_days_valid: 365
4

```

- ansible/roles/ssl/tasks/main.yml:
 - name: Create SSL private directory
 - file:
 - path: /etc/ssl/private
 - state: directory
 - mode: '0700'
 - name: Create SSL certs directory
 - file:
 - path: /etc/ssl/certs
 - state: directory
 - mode: '0755'
 - name: Get IMDSv2 token
 - uri:
 - url: http://169.254.169.254/latest/api/token
 - method: PUT
 - headers:
 - X-aws-ec2-metadata-token-ttl-seconds: "{{ imdsv2_token_ttl }}"
 - return_content: yes
 - register: imds_token
 - name: Get public IP
 - uri:
 - url: http://169.254.169.254/latest/meta-data/public-ipv4
 - headers:
 - X-aws-ec2-metadata-token: "{{ imds_token.content }}"
 - return_content: yes
 - register: public_ip
 - name: Save public IP as fact
 - set_fact:
 - server_public_ip: "{{ public_ip.content }}"
 - name: Generate self-signed certificate
 - command: >
 - openssl req -x509 -nodes -days {{ ssl_days_valid }}
 - newkey rsa:2048
 - keyout /etc/ssl/private/selfsigned.key
 - out /etc/ssl/certs/selfsigned.crt
 - subj "/CN={{ server_public_ip }}"
 - addext "subjectAltName=IP:{{ server_public_ip }}"
 - args:
 - creates: /etc/ssl/certs/selfsigned.crt

- Save screenshot as: task13_ssl_tasks_main.png – content of roles/ssl/tasks/main.yml.

```

1  ---
2  - name: Create SSL private directory
3    file:
4      path: /etc/ssl/private
5      state: directory
6      mode: '0700'
7
8  - name: Create SSL certs directory
9    file:
10     path: /etc/ssl/certs
11     state: directory
12     mode: '0755'
13
14  - name: Get IMDSv2 token
15    uri:
16     url: http://169.254.169.254/latest/api/token
17     method: PUT
18     headers:
19       X-aws-ec2-metadata-token-ttl-seconds: "{{ imdsv2_token_ttl }}"
20     return_content: yes
21     register: imds_token
22
23  - name: Get public IP
24    uri:
25     url: http://169.254.169.254/latest/meta-data/public-ipv4

```

9. Role: webapp

- ansible/roles/webapp/defaults/main.yml:

nginx_user: nginx

nginx_worker_processes: auto

nginx_worker_connections: 1024

nginx_error_log_level: notice

Webapp settings

web_root: /usr/share/nginx/html

web_index_file: index.php

- Save screenshot as: task13_webapp_defaults_main.png – content of roles/webapp/defaults/main.yml.

```

1  ---
2  nginx_user: nginx
3  nginx_worker_processes: auto
4  nginx_worker_connections: 1024
5  nginx_error_log_level: notice
6
7  # Webapp settings
8  web_root: /usr/share/nginx/html
9  web_index_file: index.php
10

```

- ansible/roles/webapp/files/index.php - PHP metadata page.
<?php

```

// Get hostname
$hostname = gethostname();

// Deployment date
$deployed_date = date("Y-m-d H:i:s");

// Metadata base URL
$metadata_base = "http://169.254.169.254/latest/";

// Function to get IMDSv2 token
function getImdsV2Token() {
    $sch = curl_init("http://169.254.169.254/latest/api/token");
    curl_setopt_array($sch, [
        CURLOPT_RETURNTRANSFER => true,
        CURLOPT_CUSTOMREQUEST => "PUT",
        CURLOPT_HTTPHEADER => [
            "X-aws-ec2-metadata-token-ttl-seconds: 21600"
        ],
        CURLOPT_TIMEOUT => 2
    ]);

    $token = curl_exec($sch);
    curl_close($sch);

    return $token ?: null;
}

// Function to fetch metadata using token
function getMetadata($path, $token) {
    $url = "http://169.254.169.254/latest/meta-data/" . $path;

    $sch = curl_init($url);
    curl_setopt_array($sch, [
        CURLOPT_RETURNTRANSFER => true,
        CURLOPT_HTTPHEADER => [
            "X-aws-ec2-metadata-token: $token"
        ],
        CURLOPT_TIMEOUT => 2
    ]);

    $value = curl_exec($sch);
    curl_close($sch);

    return $value ?: "N/A";
}

// Fetch token
$token = getImdsV2Token();

// Fetch metadata only if token is available
$instance_id = $token ? getMetadata("instance-id", $token) : "N/A";
$private_ip = $token ? getMetadata("local-ipv4", $token) : "N/A";
$public_ip = $token ? getMetadata("public-ipv4", $token) : "N/A";
$public_dns = $token ? getMetadata("public-hostname", $token) : "N/A";
?>

```

```

<!DOCTYPE html>
<html>
<head>
  <title>Frontend Web Server</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 50px;
      background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
      color: white;
    }
    .container {
      background: rgba(255, 255, 255, 0.1);
      padding: 30px;
      border-radius: 10px;
      box-shadow: 0 8px 32px rgba(31, 38, 135, 0.37);
    }
    h1 {
      color: #fff;
      text-shadow: 2px 2px 4px rgba(0,0,0,0.3);
    }
    .info {
      margin: 15px 0;
      padding: 10px;
      background: rgba(255,255,255,0.2);
      border-radius: 5px;
    }
    .label {
      font-weight: bold;
      color: #ffd700;
    }
    .info a {
      color: white; /* same as other values */
      text-decoration: none; /* remove underline */
      font-weight: normal;
    }

    .info a:hover {
      text-decoration: underline; /* optional: underline on hover */
    }
  </style>
</head>
<body>
  <div class="container">
    <h1>🚀 Nginx Front End Web Server </h1>

    <div class="info"><span class="label">Hostname:</span> <?=
htmlspecialchars($hostname) ?></div>
    <div class="info"><span class="label">Instance ID:</span> <?=
htmlspecialchars($instance_id) ?></div>
    <div class="info"><span class="label">Private IP:</span> <?=
htmlspecialchars($private_ip) ?></div>
    <div class="info"><span class="label">Public IP:</span> <?=
htmlspecialchars($public_ip) ?></div>
    <div class="info"><span class="label">Public DNS:</span>

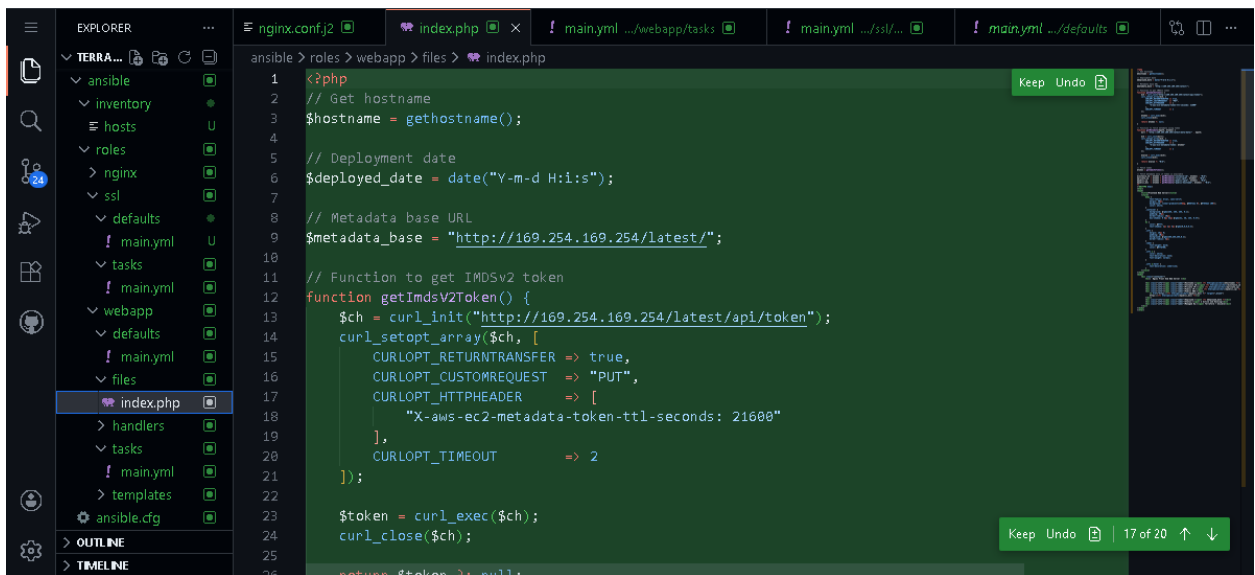
```

```

<a href="https://<?= htmlspecialchars($public_dns) ?>" target="_blank">
https://<?= htmlspecialchars($public_dns) ?></a>
</div>
<div class="info"><span class="label">Deployed:</span> <?= $deployed_date ?></div>
<div class="info"><span class="label">Status:</span> ☒ Active and Running</div>
<div class="info"><span class="label">Managed By:</span> Terraform + Ansible</div>
</div>
</body>
</html>

```

- Save screenshot as: task13_webapp_files_index_php.png – content of roles/webapp/files/index.php.

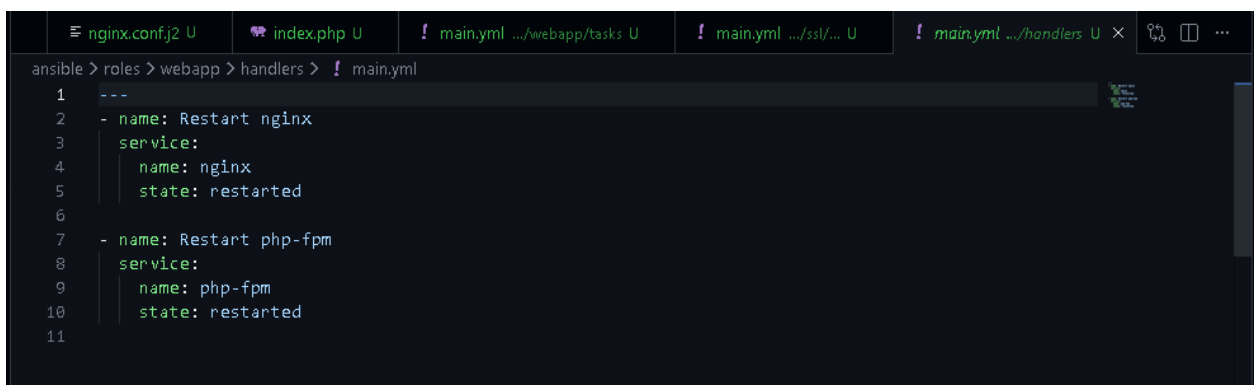


```

1 <?php
2 // Get hostname
3 $hostname = gethostname();
4
5 // Deployment date
6 $deployed_date = date("Y-m-d H:i:s");
7
8 // Metadata base URL
9 $metadata_base = "http://169.254.169.254/latest/";
10
11 // Function to get IMDSv2 token
12 function getImdsV2Token() {
13     $ch = curl_init("http://169.254.169.254/latest/api/token");
14     curl_setopt_array($ch, [
15         CURLOPT_RETURNTRANSFER => true,
16         CURLOPT_CUSTOMREQUEST => "PUT",
17         CURLOPT_HTTPHEADER => [
18             "X-aws-ec2-metadata-token-ttl-seconds: 21600"
19         ],
20         CURLOPT_TIMEOUT => 2
21     ]);
22
23     $token = curl_exec($ch);
24     curl_close($ch);
25
26     return $token ?> curl_*

```

- ansible/roles/webapp/handlers/main.yml:
 - name: Restart nginx
 - service:
 - name: nginx
 - state: restarted
 - name: Restart php-fpm
 - service:
 - name: php-fpm
 - state: restarted
- Save screenshot as: task13_webapp_handlers_main.png – content of roles/webapp/handlers/main.yml.



```

1 ---
2 - name: Restart nginx
3   service:
4     name: nginx
5     state: restarted
6
7 - name: Restart php-fpm
8   service:
9     name: php-fpm
10    state: restarted
11

```

- ansible/roles/webapp/templates/nginx.conf.j2:


```

user {{ nginx_user }};
worker_processes {{ nginx_worker_processes }};
error_log /var/log/nginx/error.log {{ nginx_error_log_level }};
pid /run/nginx.pid;

events {
    worker_connections {{ nginx_worker_connections }};
}

http {
    log_format main '$remote_addr - $remote_user [$time_local] "$request"'
        '$status $body_bytes_sent "$http_referer"'
        '"$http_user_agent" "$http_x_forwarded_for"';

    access_log /var/log/nginx/access.log main;

    sendfile          on;
    tcp_nopush        on;
    keepalive_timeout 65;
    types_hash_max_size 4096;

    include           /etc/nginx/mime.types;
    default_type      application/octet-stream;

    upstream backend_servers {
        server 158.252.94.241:80;
        server 158.252.94.242:80 backup;
    }

    server {
        listen 443 ssl;
        server_name {{ server_public_ip }};

        ssl_certificate /etc/ssl/certs/selfsigned.crt;
        ssl_certificate_key /etc/ssl/private/selfsigned.key;

        location / {
            root {{ web_root }};
            index {{ web_index_file }} index.html index.htm;
            # proxy_pass http://158.252.94.241:80;
            # proxy_pass http://backend_servers;

            location / {
                try_files $uri $uri/ =404;
            }

            # ● This block is necessary for Php Website
            location ~ \.php$ {
                include fastcgi_params;
                fastcgi_pass unix:/run/php-fpm/www.sock;
                fastcgi_index index.php;
                fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
            }
        }
    }

```

```

}

server {
    listen 80;
    server_name _;
    return 301 https://$host$request_uri;
}
}

```

- Save screenshot as: task13_webapp_templates_nginx_conf.png – content of roles/webapp/templates/nginx.conf.j2.

```

1 user {{ nginx_user }};
2 worker_processes {{ nginx_worker_processes }};
3 error_log /var/log/nginx/error.log {{ nginx_error_log_level }};
4 pid /run/nginx.pid;
5
6
7 events {
8     worker_connections {{ nginx_worker_connections }};
9 }
10
11 http {
12     log_format main '$remote_addr - $remote_user [$time_local] "$request"'
13                     '$status $body_bytes_sent "$http_referer"'
14                     '"$http_user_agent" "$http_x_forwarded_for"';
15
16     access_log /var/log/nginx/access.log main;
17
18     sendfile        on;
19     tcp_nopush      on;
20     keepalive_timeout 65;
21     types_hash_max_size 4096;
22
23     include          /etc/nginx/mime.types;
24     default_type      application/octet-stream;
25
26     upstream backend_servers {
27         server 158.252.94.241:80;
28     }
29 }

```

- ansible/roles/webapp/tasks/main.yml:
 - name: Install PHP packages
 - yum:
 - name:
 - php-fpm
 - php-curl
 - state: present
 - notify: Restart php-fpm
 - name: Copy PHP website
 - copy:
 - src: index.php
 - dest: "{{ web_root }}/{{ web_index_file }}"
 - owner: nginx
 - group: nginx
 - mode: '0644'
 - notify: Restart nginx
 - name: Deploy nginx config
 - template:
 - src: nginx.conf.j2
 - dest: /etc/nginx/nginx.conf
 - notify: Restart nginx
 - name: Start and enable php-fpm
 - service:

```
name: php-fpm
state: started
enabled: true
```

- Save screenshot as: task13_webapp_tasks_main.png – content of roles/webapp/tasks/main.yml.

```
1 ---
2 - name: Install PHP packages
3   yum:
4     name:
5       - php-fpm
6       - php-curl
7     state: present
8     notify: Restart php-fpm
9
10 - name: Copy PHP website
11   copy:
12     src: index.php
13     dest: "{{ web_root }}/{{ web_index_file }}"
14     owner: nginx
15     group: nginx
16     mode: '0644'
17   notify: Restart nginx
18
19 - name: Deploy nginx config
20   template:
21     src: nginx.conf.j2
22     dest: /etc/nginx/nginx.conf
23   notify: Restart nginx
24
25 - name: Start and enable php-fpm
26   service:
27     name: php-fpm
28     state: started
29     enabled: true
```

10. Final role-based playbook ansible/my-playbook.yml:

```
---
```

```
- name: Deploy NGINX Web Stack with SSL and PHP
hosts: nginx
become: true
roles:
  - nginx
  - ssl
  - webapp
```

- Save screenshot as: task13_my_playbook_roles.png – final my-playbook.yml referencing three roles.

```
1 ---
2 - name: Deploy NGINX Web Stack with SSL and PHP (using roles)
3   hosts: nginx
4   become: true
5   roles:
6     - nginx
7     - ssl
8     - webapp
9
```

11. Run:

```
ansible-playbook -i inventory/hosts my-playbook.yml
```

- Save screenshot as: task13_ansible_play_roles.png – playbook output with all roles executed.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
@Hamail-maker → /workspaces/terraform_machine/ansible (main) $ ansible-playbook -i inventory/hosts my-playbook.yaml

TASK [ssl : Save public IP as fact] *****
ok: [158.252.93.66]

TASK [ssl : Generate self-signed certificate] *****
ok: [158.252.93.66]

TASK [webapp : Install PHP packages] *****
ok: [158.252.93.66]

TASK [webapp : Copy PHP website] *****
ok: [158.252.93.66]

TASK [webapp : Deploy nginx config] *****
ok: [158.252.93.66]

TASK [webapp : Start and enable php-fpm] *****
ok: [158.252.93.66]

PLAY RECAP *****
158.252.93.66 : ok=14 changed=0 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0

@Hamail-maker → /workspaces/terraform_machine/ansible (main) $
```

Visit <https://<public-ip>> and verify the PHP page with metadata.

- Save screenshot as: task13_php_https_browser_roles.png – browser showing final PHP page via HTTPS.

